INGÉNIERIE LOGICIELLE / GUIDES D’UTILISATION

ArquitecturaRepo

Manuel d’utilisation complet

VERSION 1.4 / FRANÇAIS

Huit chapitres : découverte, installation, adoption, opération, changements, récupération, acceptation et exemple de produit.

MATÍAS ACUÑA · NOVEMBRE 2026

Documentation vérifiée par rapport aux sources d’ArquitecturaRepo.

Table des matières

00 — Avant de commencer **5**
Dans ce chapitre . . . 5
1. Ce que vous installez et pourquoi . . . 5
2. Qui prend quelles décisions . . . 6
3. Les informations qui doivent survivre au chat . . . 7
4. Comment lire le manuel complet . . . 7
5. Préparer les décisions du CLIENT . . . 8
6. Périmètre, limites et versions . . . 8
7. Un parcours de développement concret . . . 9
8. Questions fréquentes avant de commencer . . . 9
9. Une fiche de départ d'une page . . . 10
10. Comment évaluer la valeur du système . . . 10
Sources et portée . . . 11

01 — Installation et configuration **12**
Dans ce chapitre . . . 12
1. Environnement et comptes . . . 12
2. Starter, fork et application : trois emplacements distincts . . . 13
3. Vérifier les chemins avant toute écriture . . . 13
4. Choisir un profil proportionné . . . 14
5. Créer la couche de coordination . . . 14
6. Valider sans inventer une acceptation . . . 15
7. La prochaine étape est la découverte du produit . . . 15
8. Avant de recommencer une installation . . . 16
9. Du scaffold à une adoption réelle . . . 16
10. Connexions, permissions et confidentialité . . . 17
11. Diagnostiquer sans détruire le contexte . . . 17
Sources et portée . . . 17

02 — Le Planner et l'adoption du projet **19**
Dans ce chapitre . . . 19
1. Parler du produit avant de parler des agents . . . 19
2. Préparer le contexte de travail . . . 20
3. Découverte : poser les questions qui comptent . . . 20
4. Baseline du produit et plan de livraison . . . 21
5. L'adoption comme lien avec le projet réel . . . 21
6. L'épreuve du contexte frais . . . 22
7. Autoriser la première campagne . . . 22
8. Exemple de session avec le Planner . . . 23
9. Un baseline que le CLIENT peut confirmer . . . 23
10. Délimiter une première capacité . . . 23
Sources et portée . . . 24

03 — Fonctionnement quotidien **25**
Dans ce chapitre . . . 25
1. Reprendre le travail sans repartir de zéro . . . 25
2. Transformer une demande en capacité vérifiable . . . 25
3. Responsabilité continue du Controller . . . 26

4. Autorisations et effets externes . . . 26
5. Lire un rapport d'exécution de manière critique . . . 27
6. Git, changements locaux et continuité . . . 27
7. Ressources, artefacts et espace disque . . . 28
8. Coordination, concurrence et rôle du CLIENT . . . 28
9. Revue indépendante du candidat exact . . . 29
10. Une routine quotidienne simple . . . 29
Sources et portée . . . 30

04 — Demander des changements et de nouvelles capacités 31
Dans ce chapitre . . . 31
1. Décrire le résultat recherché . . . 31
2. Distinguer fonctionnalité nouvelle, bogue et refactorisation . . . 32
3. Un format simple pour exprimer une demande . . . 32
4. Priorités, coût et chemin critique . . . 33
5. Changements sensibles concernant données et sécurité . . . 33
6. Retour utilisateur et évaluation perceptuelle . . . 34
7. Renvoyer les défauts au même Controller . . . 34
8. Éviter les politiques et campagnes doublonnées . . . 34
9. Exemples de demandes bien délimitées . . . 35
10. Confirmer que la nouvelle version est actuelle . . . 35
Sources et portée . . . 36

05 — Erreurs, incidents et récupération 37
Dans ce chapitre . . . 37
1. Conserver les preuves avant de réparer . . . 37
2. Pannes d'environnement, de comptes et de permissions . . . 38
3. Lorsque la validation de contrôle échoue . . . 38
4. Git ne sert pas à effacer les problèmes . . . 39
5. Défauts répétés et causes systémiques . . . 39
6. Sécurité, confidentialité et droits externes . . . 40
7. Prouver la fin de l'incident . . . 40
8. Trois scénarios de récupération . . . 41
9. Opérations sensibles et autorisations . . . 41
10. Ce qu'un test vert ne prouve pas . . . 42
11. Retours d'architecture et données locales . . . 42
Sources et portée . . . 42

06 — Clôture des capacités et acceptation 44
Dans ce chapitre . . . 44
1. Une capacité est un résultat de produit . . . 44
2. Les cinq composantes d'une clôture . . . 44
3. Le candidat exact est l'unité de revue . . . 45
4. Preuves insuffisantes sans campagnes artificielles . . . 46
5. Accepter, reprendre ou changer d'objectif . . . 46
6. Revues périodiques et règle d'arrêt . . . 47
7. Un enregistrement de clôture vérifiable . . . 47
8. Exemple de rejet et de correction . . . 48
9. Produit utilisable contre framework vert . . . 48
10. Que faire après la clôture . . . 49
Sources et portée . . . 49

07 — Exemple guidé : une application de gestion pour un club 50
Dans ce chapitre 50
1. Le produit demandé par le club 50
2. Découvrir les questions qui changent le résultat 51
3. Installer la couche de contrôle, pas l'application 51
4. Première capacité : gestion des membres 52
5. Du Planner à I3 : un ordre de travail complet 52
6. Ce qu'il faut tester avant d'accepter 53
7. Deuxième capacité : cotisations et paiements 54
8. Évaluer l'ensemble du parcours 55
9. Premier cycle avec des données fictives 55
10. Enregistrer et acheminer un défaut 55
11. Les dépendances du deuxième cycle 56
12. Exercice final : récupérer le projet 56
Sources et portée 57

00 — Avant de commencer

ArquitecturaRepo · Manuel d’utilisation complet · Édition v1.4 · Français

> ArquitecturaRepo est une architecture de coordination pour le développement logiciel, pas une application prête à l’emploi ni un programmeur automatique. Sa valeur consiste à rendre les décisions, les responsabilités, l’état courant et les preuves **reconstructibles au-delà d’une conversation**. Pour les commandes et le périmètre effectivement testés dans la version publique, consultez le guide de démarrage v1.4 et les notes de version.

Dans ce chapitre

1. Ce que vous installez et pourquoi
2. Qui prend quelles décisions
3. Les informations qui doivent survivre au chat
4. Comment lire le manuel complet
5. Préparer les décisions du CLIENT
6. Périmètre, limites et versions
7. Un parcours de développement concret
8. Questions fréquentes avant de commencer
9. Une fiche de départ d’une page
10. Comment évaluer la valeur du système

1. Ce que vous installez et pourquoi

Imaginez un projet développé avec un assistant de programmation. Un jour, l’assistant crée une API, corrige un problème et propose une organisation. Le lendemain, une nouvelle session s’ouvre et personne ne sait exactement pourquoi telle décision a été prise, quelle version du code correspond à la dernière revue ou quel résultat le propriétaire du produit avait accepté. Le code existe, mais son contexte opérationnel reste fragile.

ArquitecturaRepo vise ce problème de continuité. Il place les décisions durables et leurs références dans le dépôt et dans des surfaces canoniques explicitement identifiées. Les exigences du produit, l’identité des sources, les responsabilités, les campagnes d’ingénierie, les revues et les preuves sont liées de manière vérifiable. Une conversation reste une interface pratique, mais elle ne devient pas la seule mémoire du projet.

L’installation initiale peut créer un dossier `.agentic-sdlc/` dans un dépôt de produit qui ne possède pas déjà une gouvernance canonique incompatible. Cette couche contient des contrats, des modèles d’état, des protocoles et des outils de validation. Elle ne construit pas automatiquement votre application, ne configure pas votre base de production et n’obtient pas d’accès à un fournisseur d’IA.

Le but n’est pas d’ajouter un grand nombre d’agents. Il s’agit de réduire le temps perdu à reconstruire le contexte, de prévenir les décisions contradictoires et de vérifier les résultats avant d’annoncer qu’une fonctionnalité est prête. Une structure plus lourde que les risques qu’elle traite peut devenir contre-productive :

la proportionnalité fait partie du principe de conception.

La continuité est particulièrement importante lorsque le projet dure plusieurs semaines. Un même objectif peut impliquer du code, une base de données, des fichiers générés, des décisions humaines, des interruptions et des personnes qui ne travaillent pas au même moment. Sans provenance claire, un rapport positif sur une ancienne version peut être confondu avec l'état actuel.

2. Qui prend quelles décisions

Le **CLIENT** est la personne responsable du produit. Il décide du problème à résoudre, des priorités, des contraintes, des choix métier et des effets externes nécessitant son autorisation. Il peut demander une application tout en ne connaissant pas les subtilités de Git, des transactions SQL ou des runners. L'organisation doit protéger son temps et ne pas lui transférer les opérations techniques.

Le **I2 Planner** reconstruit la situation, confirme les exigences, définit les capacités et leurs dépendances, propose la stratégie et audite les résultats terminaux. Pour une décision stratégique importante, il examine le produit de bout en bout puis réalise un examen adversarial séparé. Il ne revient pas automatiquement au début du processus à chaque bogue ordinaire.

Le **I3 Controller** prend en charge l'exécution continue d'une campagne autorisée. Son travail comprend l'implémentation, les tests, les refactorisations internes, la résolution des défauts, la récupération des interruptions, l'intégration et la revue indépendante. Tant que le travail reste dans les limites de la campagne, il ne demande pas une nouvelle autorisation stratégique pour chaque correction.

Le **I4** peut être un spécialiste ou un réviseur indépendant pour une tâche délimitée. Les quatre niveaux de responsabilités ne signifient pas quatre instances d'IA obligatoires. Un **I1 MASTER OWNER** peut être utile lorsqu'il faut coordonner plusieurs produits ou lignes de travail, résoudre des contradictions d'ownership ou protéger les dépendances entre systèmes. Une petite application n'a pas besoin de reproduire toute cette organisation si ses risques ne le justifient pas.

Question	Autorité normale
Quel problème le produit doit-il résoudre ?	CLIENT, avec découverte guidée par le Planner.
Quelle capacité construire en premier ?	Planner selon les priorités confirmées.
Comment structurer classes, transactions ou index ?	Ingénierie sous la responsabilité de I3.
Le résultat respecte-t-il les exigences décidées ?	Tests et revue, puis audit du Planner.
L'expérience est-elle satisfaisante pour le public prévu ?	Évaluation humaine compétente, éventuellement CLIENT.
Peut-on payer, publier ou transmettre des données ?	Autorité explicite du responsable concerné.

Ces distinctions évitent deux extrêmes. L'ingénierie ne doit pas inventer une règle de produit faute de réponse, et le CLIENT ne doit pas avoir à choisir les paramètres techniques d'une requête ou à réparer manuellement un contrôleur. Chaque décision doit rester auprès de son propriétaire légitime.

3. Les informations qui doivent survivre au chat

Une nouvelle session doit pouvoir identifier les besoins confirmés, la source actuelle, les éléments de Product State, le propriétaire de chaque opération et la prochaine action autorisée. Le dépôt contient le code et les décisions versionnées. La base de données peut contenir des faits vivants. Des manifests et des artefacts démontrent les résultats. Des journaux de transaction permettent de comprendre une interruption. Ces surfaces ont des rôles différents.

Une simple déclaration «terminé» ne remplace pas la preuve. Si un réviseur examine le candidat A et qu'un agent modifie ensuite le code pour produire B, la revue de A ne certifie pas B. La même branche Git et un nom de fichier inchangé ne suffisent pas à garantir l'identité. Les références doivent pointer vers le candidat effectivement examiné.

Le même problème existe pour les données. Une base SQLite en mode WAL peut avoir des modifications confirmées qui ne se trouvent pas encore dans le fichier principal. Un hash de ce seul fichier n'atteste pas nécessairement l'état transactionnel complet. Selon le risque, il faut un témoin cohérent ou une incertitude explicitement enregistrée.

Les décisions de produit, l'état courant mutable, l'autorité de campagne et les journaux de runtime doivent rester séparés. Si plusieurs fichiers prétendent être la source de vérité actuelle pour le même sujet, le prochain agent risque de choisir le mauvais. Une bonne architecture possède un propriétaire et une surface canonique pour chaque préoccupation.

La mémoire de conversation reste utile pour retrouver une piste, mais elle peut être incomplète ou périmée. En cas de contradiction, la source physique vivante et les enregistrements canoniques priment. Si une incertitude matérielle ne peut pas être résolue, l'action dépendante reste en HOLD, tandis que des vérifications de lecture peuvent continuer.

4. Comment lire le manuel complet

Le livre comprend huit chapitres détaillés. Le chapitre 00 explique le cadre. Le chapitre 01 traite de l'installation. Le chapitre 02 présente le Planner, la découverte des besoins et l'adoption. Le chapitre 03 décrit le travail quotidien. Le chapitre 04 explique comment demander des changements. Le chapitre 05 aborde les erreurs et la récupération. Le chapitre 06 distingue les preuves et l'acceptation. Le chapitre 07 suit un exemple fictif de gestion d'un club.

Pour un premier essai rapide, utilisez le miniguide français. Il est volontairement court et ne constitue pas ce manuel complet. La FAQ française répond à des questions de démarrage. Les documents de référence du dépôt servent à vérifier des comportements et contrats précis.

Il n'est pas indispensable de mémoriser tous les noms de fichiers internes. Il faut surtout apprendre à poser les bonnes questions : que souhaite la personne utilisatrice, où est la preuve, qui peut agir, quelle version est actuelle et qu'est-ce qui empêcherait une acceptation honnête ? La pratique avec un produit concret donne du sens à ces concepts.

Les commandes figurant dans les chapitres sont des exemples adaptés à v1.4. Les interfaces et les outils peuvent évoluer. Avant une action importante, vérifiez les options du script de la version effectivement utilisée et ne mélangez pas des guides

historiques avec une distribution plus récente sans contrôler les différences.

5. Préparer les décisions du CLIENT

Avant la première campagne, une description d'une page peut suffire : qui utilisera le produit, pour quelle tâche, quel premier résultat observable, avec quels délais et quel budget. Mentionnez les données sensibles, les contraintes d'accès et les opérations qui devront être explicitement autorisées. Les inconnues peuvent rester inconnues si elles sont clairement identifiées.

Par exemple, «je veux gérer les membres d'un club» ne précise pas si une cotisation impayée bloque l'inscription à une activité, si une désactivation efface l'historique ou si plusieurs employés travaillent simultanément. Le Planner doit confirmer ces règles, car elles changent la fonctionnalité. En revanche, le nom d'une classe ou le choix d'un index SQL appartient normalement à l'ingénierie.

Un bon premier objectif est observable : «Un administrateur peut créer un membre fictif, le retrouver et voir que ses modifications persistent après redémarrage». Cette phrase donne une direction à l'implémentation et des critères de test. «Construire une architecture professionnelle» n'indique pas à lui seul comment mesurer la livraison.

Le CLIENT peut apprendre en voyant un prototype puis ajuster son besoin. Les décisions nouvelles doivent être enregistrées et leurs conséquences évaluées. Le système n'est pas conçu pour rendre irrévocable la première intuition, mais pour éviter que plusieurs versions contradictoires de la même règle restent actives.

6. Périmètre, limites et versions

La version publique v1.4.0 existe sous forme de distribution versionnée, avec archive portable, checksum et manifest. Pour une installation reproductible, contrôlez les octets téléchargés contre cette publication. La branche `main` peut avoir reçu de la documentation après la sortie : son contenu ne fait pas rétroactivement partie de l'archive publiée.

Les mécanismes portables testés comprennent les profils, le scaffold, la validation, la récupération et des parcours synthétiques sur Linux et Windows. Cette vérification ne démontre pas qu'une application particulière est correcte ni qu'une intégration native à Codex Desktop ou à un autre service a été exercée. Les outils externes ont leurs propres autorisations et tests.

ArquitecturaRepo est indépendant du fournisseur de modèle. Il n'installe pas un LLM, ne configure pas de facturation et n'autorise pas des effets de réseau par défaut. Il peut aider à coordonner une équipe humaine, des agents facultatifs ou un mélange des deux. Les résultats dépendent de l'implémentation et des preuves effectives, non du seul nom de l'architecture.

Les profils `SMALL`, `STATEFUL`, `ARTIFACT_HEAVY`, `MULTI_WORKSTREAM`, `MULTI_PRODUCT` et `HIGH_CONSEQUENCE` permettent de proportionner les contrôles. Ils ne constituent pas des niveaux de certification ni des obligations de créer davantage d'agents. Le profil est choisi selon le risque réel du produit et peut être réévalué si ce risque change.

Une application qui possède déjà une gouvernance canonique ne doit pas être écrasée par le scaffold générique. Le précontrôle en lecture seule permet d'examiner les sources et témoins requis. Il ne constitue pas une migration universelle d'un Product State v1.2/v1.3 ni une permission de remplacer des données existantes.

7. Un parcours de développement concret

Prenons une petite application de réservation. Le CLIENT demande la création et la consultation de réservations, avec impossibilité de dépasser la capacité d'une salle. Le Planner vérifie les utilisateurs, les conflits, les annulations, les données et le délai. Les décisions confirmées forment un baseline de produit lié au code et à l'environnement actuels.

Le Planner définit une première capacité et un charter. I3 implémente les interfaces, corrige les bogues, intègre un candidat et exécute les contrôles applicables. L'essai physique consiste à créer une réservation, relire le résultat, tenter un conflit et vérifier la persistance si elle fait partie des exigences. Le rapport doit indiquer la version exacte testée.

Le Planner audite ensuite les preuves. Le CLIENT juge les décisions réservées, notamment l'expérience utilisateur. Si une double réservation est possible alors qu'elle est interdite, I3 la corrige dans la campagne actuelle. Si le CLIENT introduit ensuite une priorité réservée à certains membres, c'est une règle nouvelle que le Planner doit confirmer et intégrer à la stratégie.

Après acceptation, les références canoniques indiquent le candidat réellement approuvé. Une personne qui rejoint le projet demain peut retrouver les exigences, l'état et la prochaine action sans relire toute la discussion. Cette continuité est l'un des avantages potentiels d'une architecture de développement structurée.

8. Questions fréquentes avant de commencer

Faut-il plusieurs agents ? Non. Les responsabilités peuvent être assumées par un petit nombre de personnes ou d'outils, à condition de conserver les limites d'autorité et les preuves nécessaires. Plus d'agents peut au contraire augmenter les conflits si les ressources partagées ne sont pas coordonnées.

Est-ce mieux que coder directement dans un chat ? Cela dépend du contexte. Une petite démonstration peut être plus rapide en dialogue direct. Un produit long et évolutif peut bénéficier d'une meilleure conservation des décisions et des preuves. Le gain doit être évalué en temps, qualité, erreurs, récupérabilité et effort demandé au CLIENT ; il ne faut pas proclamer une supériorité non mesurée.

Que faire lorsqu'un agent se trompe ? Si le défaut relève du charter approuvé, I3 corrige et reteste. La difficulté technique ne justifie pas automatiquement un nouveau plan stratégique. Une vraie modification du besoin ou une invalidation matérielle peut en revanche justifier une nouvelle décision du Planner.

Que signifie HOLD ? C'est l'interdiction temporaire d'une action dépendante lorsque les preuves ou autorisations sont insuffisantes. Cela n'empêche pas les diagnostics sûrs en lecture seule. Un HOLD utile précise la cause, le propriétaire et les informations à obtenir.

Le dépôt de mon application peut-il rester privé ? Oui. Le starter n'exige pas l'envoi du code, des données, des identifiants ou des conversations à son mainteneur. Les intégrations externes ont leurs propres réglages et doivent respecter les permissions et la confidentialité.

9. Une fiche de départ d’une page

Avant une campagne, essayez de répondre à ces questions. Les réponses doivent pointer vers les sources véritables du projet, pas vers un document parallèle devenu concurrent :

Sujet	Ce qui doit être connu
Produit	Public, problème et premier résultat observable.
Dépôt	Source réelle, branche/ref, modifications locales et historique pertinent.
Responsabilités	CLIENT, Planner, Controller et propriétaires des ressources partagées.
Exigences	Décisions confirmées, questions ouvertes et exclusions.
Environnement	Stack, base, réseau, contraintes de sécurité, coûts et accès.
État courant	Product State et références sans contradiction matérielle.
Campagne	Objectif autorisé, interfaces, critères et falsificateurs.
Preuves	Gates nécessaires, identité du candidat et résultats vérifiés.
Continuité	Prochaine action reconstructible dans une session nouvelle.

Si une donnée essentielle manque, identifiez qui peut la confirmer. Un identifiant de commit absent est un problème de traçabilité, pas une préférence métier. Une règle d’accès non décidée relève du CLIENT. Une erreur de test relève généralement du Controller. Les bonnes routes réduisent les discussions inutiles.

Cette fiche sert d’index, pas de nouvelle surface `CURRENT`. Dupliquer la même vérité dans plusieurs fichiers sans ownership clair accroît le risque de divergence. Le principe reste un état canonique par sujet, avec des références qui permettent de comprendre ses dépendances.

10. Comment évaluer la valeur du système

Après plusieurs semaines, mesurez si le projet reprend facilement après une interruption, si les décisions restent cohérentes, si les défauts sont résolus au bon niveau et si les personnes utilisatrices disposent d’une capacité démontrable. La quantité de commits, de réunions et de tokens ne constitue pas une mesure directe du produit.

Pour comparer à une approche de programmation directe, observez le temps total, l’effort du CLIENT, la qualité des tests, les régressions, la continuité et la facilité de transférer le travail. L’architecture peut imposer un coût initial plus important ; ce coût est justifié seulement s’il améliore réellement la livraison ou réduit un risque important.

N’ajoutez pas un nouveau protocole par réflexe après chaque problème. Une règle supplémentaire doit traiter une catégorie de défaillances ou une capacité qui manque

vraiment. L'efficacité du système tient à la qualité des décisions et de leur exécution, pas à la taille de sa documentation.

Suite : chapitre 01, installation et configuration de la couche de contrôle v1.4.

Sources et portée

- Vue d'ensemble de l'architecture
- Profils de déploiement
- Limite de décision du CLIENT
- Cycle de vie des capacités
- Démarrage public v1.4
- Notes de version v1.4

Ce chapitre présente une méthode et ses limites. Il ne certifie pas des applications tierces, des intégrations externes ni des performances comparatives non mesurées.

01 — Installation et configuration

ArquitecturaRepo · Manuel d’utilisation complet · Édition v1.4 · Français

> Installer le starter crée une première couche de coordination ; **cela n’adopte pas automatiquement l’application** et ne démontre pas qu’un produit fonctionne. Les commandes suivent le guide public v1.4. L’état initial `HOLD / UNTRUSTED_CONTEXT` est normal tant que les exigences, les sources et les responsabilités ne sont pas établies.

Dans ce chapitre

1. Environnement et comptes
2. Starter, fork et application : trois emplacements distincts
3. Vérifier les chemins avant toute écriture
4. Choisir un profil proportionné
5. Créer la couche de coordination
6. Valider sans inventer une acceptation
7. La prochaine étape est la découverte du produit
8. Avant de recommencer une installation
9. Du scaffold à une adoption réelle
10. Connexions, permissions et confidentialité
11. Diagnostiquer sans détruire le contexte

1. Environnement et comptes

Le parcours portable v1.4 repose sur Python 3.12, un accès aux fichiers, et idéalement Git. Python 3.12 est la version testée par la CI de cette release. Un éditeur de code peut faciliter l’inspection, mais une connexion à un fournisseur d’IA ou à un environnement de développement particulier n’est pas nécessaire pour créer et vérifier la couche de contrôle.

Sous Windows, ouvrez PowerShell et vérifiez les exécutables disponibles. Les exemples sont donnés avec des guillemets pour que les chemins contenant des espaces restent explicites :

```
& "python" --version
& "python" -m pip --version
& "git" --version
```

Si une commande n’existe pas, vérifiez l’installation, la variable PATH et le terminal utilisé. Une version de Python ouverte par un éditeur peut être différente de celle du shell. Il est préférable de résoudre le problème d’environnement plutôt que de modifier un état de projet pour obtenir artificiellement un contrôle vert.

Sur Linux ou macOS, la commande peut être `python3`. Vérifiez qu’elle désigne l’interpréteur compatible et utilisez les séparateurs de chemins attendus par votre shell. Le nom d’un script présent dans l’archive ne doit pas être deviné : consultez le dépôt et l’aide de la version réellement téléchargée.

GitHub facilite la conservation des décisions et du code, mais il ne faut pas donner plus de permissions que nécessaire. Lire un dépôt, créer un commit, modifier

une branche protégée, publier une release et accéder à des données privées sont des opérations différentes. L'installation du starter ne vous donne aucune de ces autorisations automatiquement.

2. Starter, fork et application : trois emplacements distincts

Le dépôt ArquitecturaRepo contient le framework de coordination : scripts, protocoles, schémas, exemples et documentation. Un fork peut servir de copie personnelle du framework. Le projet de destination est autre chose : l'application réelle que vous développez, avec son propre code, sa base, ses utilisateurs et ses tests.

Le scaffold part de la source du framework et crée `.agentic-sdlc/` dans le dossier de destination lorsqu'aucun contrôle canonique incompatible n'existe. Il ne transforme pas par magie l'exemple synthétique en application et ne doit pas écraser le code déjà présent. Il ajoute des structures de coordination pour guider les étapes suivantes.

Emplacement	Fonction
`ArquitecturaRepo/`	Source du starter, installateur, tests et guides.
Fork du starter	Copie versionnée et éventuellement personnalisée.
`mon-projet/`	Code et environnement de votre application.
`mon-projet/.agentic-sdlc/`	État et contrats de coordination propres à ce projet.

Une confusion classique consiste à lancer un agent dans le dépôt du starter en pensant qu'il travaille déjà sur l'application. Le premier dépôt peut être parfaitement validé alors que le second n'a jamais été ouvert. Le rapport d'exécution doit identifier les deux chemins et expliquer quelle opération a réellement été effectuée.

Les fixtures fournies pour les tests sont synthétiques. Elles ne possèdent pas les exigences ni les autorisations de votre produit. Ne copiez pas leur Product State ou leur base de données en pensant accélérer l'adoption d'un projet réel.

3. Vérifier les chemins avant toute écriture

Avant d'installer, identifiez le dossier courant et le répertoire cible. À la racine du starter v1.4, affichez l'aide du script :

```
& "python" "tools\scaffold_project.py" --help
```

Cette action est en lecture seule. Elle permet de vérifier la version et les options disponibles. Si le script n'est pas trouvé, il faut d'abord savoir dans quel dépôt on se trouve et quelle archive a été extraite. Copier un script d'une version différente dans un projet au hasard n'est pas une réparation fiable.

Un chemin relatif comme `"..\mon-projet"` désigne un dossier voisin, pas un emplacement universel. Le programme doit pouvoir distinguer la source de l'installateur, la destination, les fichiers existants et les limites des chemins liés. Examinez les répertoires avant d'utiliser des commandes capables de créer ou modifier des fichiers.

Pour une installation reproductible, téléchargez le GitHub Release v1.4.0 et vérifiez le checksum SHA-256 du ZIP portable. Un checkout du tag est également une source versionnée. En revanche, `main` évolue : les guides ajoutés plus tard ne sont pas nécessairement contenus dans l'archive v1.4.0 déjà publiée.

Si une application possède déjà `.agentic-sdlc/`, Product State, campagnes ou données canoniques, arrêtez l'installation générique. Le précontrôle en lecture seule des projets gouvernés existe précisément pour éviter d'écraser cette autorité.

4. Choisir un profil proportionné

ArquitecturaRepo propose six profils. Ils orientent les familles de contrôles, sans créer des autorisations ni garantir la réussite de l'application :

Profil	Quand l'envisager
`SMALL`	Petit produit dont les données et risques restent modestes.
`STATEFUL`	Données persistantes, utilisateurs, schémas, sauvegardes et récupération.
`ARTIFACT_HEAVY`	Nombreux fichiers, rendus coûteux, provenance, licences et réutilisation.
`MULTI_WORKSTREAM`	Plusieurs chantiers alimentent le même produit final.
`MULTI_PRODUCT`	Plusieurs produits distincts partagent des ressources ou politiques.
`HIGH_CONSEQUENCE`	Risques matériels pour la sécurité, la confidentialité ou l'intégrité.

Un premier outil sans données critiques peut utiliser `SMALL`. Une application de club qui conserve cotisations et membres pourrait justifier `STATEFUL`. Un pipeline audiovisuel avec de nombreux médias peut avoir besoin de traçabilité et de contrôle des ressources. Le profil doit correspondre aux besoins réels ; il ne s'agit ni d'une médaille ni d'une invitation à multiplier les agents.

La documentation des profils et des modes de projet explique leurs choix. Un profil n'oblige pas à héberger des microservices, à prendre un abonnement ou à installer un modèle. Les décisions sur le stack et les coûts seront confirmées dans l'adoption.

5. Créer la couche de coordination

Dans le dossier du starter, sous PowerShell, créez un contrôle initial pour un projet nouveau et non gouverné :

```
& "python" "tools\scaffold_project.py" --target "..\mon-projet" --profile "SMALL" --pr
```

Pour un exemple fictif de club avec données persistantes, on pourrait utiliser :

```
& "python" "tools\scaffold_project.py" --target "..\club-app" --profile "STATEFUL" --p
```

Adaptez chemin, identifiant et objectif au projet autorisé. Sous un shell Unix, utilisez `"tools/scaffold_project.py"` et `"../mon-projet"` avec les séparateurs appropriés. L'installateur doit préserver le code de l'application et refuser les écrasements d'une gouvernance canonique préexistante.

L'état initial attendu est `HOLD / UNTRUSTED_CONTEXT`. Le framework n'a pas encore établi quelles sont les exigences validées, la source actuelle, le propriétaire du Product State ni les campagnes approuvées. Changer un champ JSON pour afficher `READY` ne rendrait pas ces informations vraies.

L'option `--with-runtime` peut ajouter un runtime portable de référence, mais elle n'authentifie aucun fournisseur externe et ne lance pas automatiquement Codex Desktop. `--no-feedback-workflow` évite l'installation du planificateur mensuel de rapport, sans désactiver nécessairement le générateur local de métadonnées. Les deux mécanismes ont des réglages distincts.

6. Valider sans inventer une acceptation

Une fois la couche créée, placez-vous dans le dossier de **l'application cible**. Exécutez les commandes de diagnostic de l'installation v1.4 :

```
& "python" -m pip install -r ".agentic-sdlc\requirements-validation.txt"
& "python" ".agentic-sdlc\tools\project_status.py" --control-root ".agentic-sdlc"
& "python" ".agentic-sdlc\tools\discover_project.py" --project-root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py" --root ".agentic-sdlc"
```

Le premier commande installe les dépendances locales déclarées. `project_status.py` inspecte le contrôle actuel. `discover_project.py` examine des métadonnées et des hashes de fichiers dans le périmètre autorisé, sans transmettre le contenu au mainteneur. `reconstruct_context.py` vérifie ce qu'une nouvelle session peut reconstruire à partir des surfaces durables.

Le résultat peut rester en HOLD si le baseline n'a pas été adopté. C'est un résultat correct ; il indique que des décisions ou des témoins de source manquent. Le validateur de contrats ne démontre pas à lui seul que le frontend s'ouvre, que la DB contient les bons enregistrements ou que l'interface est compréhensible.

Vérifiez trois points : la couche se trouve au bon endroit, les anciens fichiers de l'application ont été préservés et les résultats de diagnostic restent disponibles pour une prochaine session. Conservez les erreurs exactes. Si une dépendance manque, installez le fichier `requirements-validation.txt` déclaré plutôt que d'altérer Product State.

Le statut de cette étape est «scaffold installé et inspecté», pas «produit prêt». L'adoption doit encore relier le contrôle générique au code réel, aux données et aux décisions du CLIENT.

7. La prochaine étape est la découverte du produit

Le Planner examine le produit : qui l'utilisera, quelles règles sont déjà confirmées, quelles données existent, quelle partie du code fonctionne et quel résultat doit être livré d'abord. Les instructions d'installation et d'adaptation, le parcours de découverte et le contrat d'adoption donnent les références correspondantes.

Il faut séparer les faits du code des décisions de produit. Une table `users` ne prouve pas que tout utilisateur peut être supprimé. Une interface de paiement ne prouve pas qu'une intégration externe est autorisée. Le Planner confronte les hypothèses à la source et demande au CLIENT les décisions qui ne peuvent pas être prises par l'ingénierie.

Une adoption complète comprend une spécification examinée, un preflight, des

sources et propriétaires cohérents, puis une reconstruction de contexte frais. C'est seulement après ces gates qu'une campagne matérielle peut recevoir son charter. Installer puis programmer immédiatement sans connaître le baseline risquerait de construire une fonctionnalité hors sujet.

La durée du discovery doit rester proportionnée. Un prototype simple peut avoir un premier objectif confirmé rapidement. Un système exposé sur internet avec données sensibles nécessite une exploration plus approfondie des interfaces, de la sécurité, de la récupération et des coûts.

8. Avant de recommencer une installation

Si `.agentic-sdlc/` existe déjà, inspectez-le avant de réessayer. Il peut contenir une autorité active, un journal de transition, un Goal, des références de DB et des artefacts nécessaires à la récupération. Une nouvelle installation par-dessus peut effacer des informations que le contrôleur aurait dû conserver.

Ne supprimez pas le projet pour «faire fonctionner le tutoriel». Ne contournez pas un refus de préservation avec une option de force. Pour un projet déjà gouverné, le bon chemin commence par un attachment de lecture et une décision explicite sur les migrations éventuelles. Ce chemin n'est pas un upgrade universel et automatique de v1.2/v1.3.

En cas d'interruption, une opération peut avoir produit un effet partiel. Il faut lire l'état réellement présent avant de répéter. Un journal inachevé ne doit pas être effacé aveuglément ; un fichier de contrôle ne doit pas être remplacé par une version vierge simplement parce que la console a échoué.

Un incident d'environnement peut être corrigé par l'ingénierie sans repasser par toute la stratégie. En revanche, deux contrôleurs qui revendiquent la même autorité ou une source de vérité conflictuelle peuvent exiger une réconciliation avec le propriétaire approprié.

9. Du scaffold à une adoption réelle

Le chemin complet comprend la découverte du code, la confirmation des besoins, le baseline de produit, le baseline technique, les jalons de livraison, la spécification d'adoption, le preflight, la promotion cohérente des références et le test de contexte frais. Chaque étape apporte des faits nécessaires à la suivante.

Le baseline technique identifie versions de code, dépendances, DB, environnement, réseau, tests et risques. Le baseline de produit identifie comportements, utilisateurs, exclusions et critères de validation. Aucun des deux ne remplace l'autre. Le Planner doit les relier sans inventer que l'application remplit déjà les exigences.

Les dépendances de capacités déterminent l'ordre des campagnes. Un club ne doit pas implémenter l'éligibilité des inscriptions à partir d'un statut de cotisation non défini. Inversement, des travaux réellement indépendants peuvent avancer en parallèle si les fichiers, données et permissions ne créent pas de risque d'interférence.

Le CLIENT peut rester concentré sur les choix fonctionnels et la validation des résultats. L'ingénierie gère les schémas, leases, checkpoints, retries et migrations. Le système échoue si l'utilisateur doit superviser manuellement chaque détail pour éviter des incohérences entre agents.

10. Connexions, permissions et confidentialité

Les connecteurs GitHub, modèles d'IA, services cloud et bases de données ne deviennent actifs que si l'environnement dispose des autorisations correspondantes. Le starter public ne contient pas de jetons. Une erreur d'authentification doit être diagnostiquée dans le périmètre concerné, sans demander des secrets en clair ni élargir aveuglément les droits.

Le générateur local de retours d'architecture manipule des métadonnées restreintes. Selon l'installation, un workflow périodique peut enregistrer des rapports dans les propres artefacts GitHub Actions du projet. Aucun endpoint de collecte externe n'est livré dans la distribution publique. `--no-feedback-workflow` désactive l'ordonnancement ; pour arrêter toute génération, réglez `enabled=false` dans le fichier local de configuration du feedback.

Consultez la politique de confidentialité et de feedback avant d'autoriser des effets sortants. Les rapports publics de bogues doivent utiliser des données fictives et des logs expurgés. N'envoyez pas les noms des clients, bases privées, chemins personnels, tokens ou conversations à un issue public.

La licence MIT du starter n'accorde pas automatiquement les droits sur les images, musiques, polices, datasets ou services tiers utilisés dans une application. Les droits réels doivent être vérifiés de manière raisonnable selon l'usage, sans généraliser des permissions inexistantes.

11. Diagnostiquer sans détruire le contexte

Si Python n'est pas reconnu, vérifiez l'exécutable. Si `jsonschema` manque, installez les dépendances déclarées. Si un chemin est introuvable, vérifiez la racine courante. Si le scaffold refuse d'écraser un contrôle existant, arrêtez l'installation générique. Si la DB ou le Product State est incohérent, conservez les références et cherchez la cause.

Un problème de Git doit être traité en préservant les modifications locales et les versions distantes. Une base SQLite/WAL nécessite un témoin cohérent. Une action interrompue exige un readback avant un retry. Le chapitre 05 détaille la récupération, les autorisations et les preuves de sortie d'incident.

Le compte rendu d'installation doit identifier source exacte, chemin de destination, profil, commandes, résultats de validation et limites ouvertes. Il ne doit pas annoncer que l'application est développée, publiée ou acceptée lorsque seul le scaffold a été créé.

Suite : chapitre 02, le Planner, la découverte et l'adoption d'un projet réel.

Sources et portée

- Guide public v1.4
- Profils
- Transfert et adoption
- Précontrôle des projets gouvernés
- Confidentialité et feedback
- Script d'installation

Ces commandes et exemples n'autorisent pas le remplacement d'un système existant

ni la facturation ou le déploiement d’un service externe.

02 — Le Planner et l’adoption du projet

ArquitecturaRepo · Manuel d’utilisation complet · Édition v1.4 · Français

> La création de `.agentic-sdlc/` n’est pas l’adoption d’un produit. Le Planner doit relier les contrats génériques à un code réel, des exigences confirmées, des propriétaires légitimes, des données actuelles et un environnement vérifiable. L’adoption doit permettre la reconstruction depuis un contexte neuf **sans inventer d’état ni doubler une gouvernance existante**.

Dans ce chapitre

1. Parler du produit avant de parler des agents
2. Préparer le contexte de travail
3. Découverte : poser les questions qui comptent
4. Baseline du produit et plan de livraison
5. L’adoption comme lien avec le projet réel
6. L’épreuve du contexte frais
7. Autoriser la première campagne
8. Exemple de session avec le Planner
9. Un baseline que le CLIENT peut confirmer
10. Délimiter une première capacité

1. Parler du produit avant de parler des agents

Une personne explique : «Je voudrais gérer les inscriptions de mon club» ou «j’ai besoin d’une API pour mon activité». Le Planner commence par comprendre les personnes utilisatrices, les tâches difficiles et le résultat que le CLIENT souhaite observer. Il ne répond pas immédiatement par une liste d’agents, de microservices ou de règles de gouvernance.

Le CLIENT n’a pas à fournir le schéma relationnel ni l’arborescence de classes. Il doit toutefois confirmer les règles qui changent le comportement du produit. Par exemple, une inscription peut-elle dépasser la capacité d’une activité ? Les paiements historiques doivent-ils rester après une désactivation ? Ces décisions ont des conséquences techniques ; elles ne peuvent pas être devinées par l’implémentation.

Une bonne première conversation clarifie le public, l’environnement, les données, les permissions, les contraintes de temps et de budget, les exclusions et le premier résultat utile. Des questions restent parfois ouvertes. Le Planner les conserve comme telles, sans convertir une possibilité en exigence approuvée.

La profondeur de cette exploration doit correspondre au risque. Un prototype local sans données sensibles peut demander une courte session. Une application publique qui traite de l’argent ou des informations personnelles exige un modèle plus précis des droits d’accès, de la récupération, des coûts et des impacts possibles.

2. Préparer le contexte de travail

Avant de modifier le projet, examinez sa source et son état. Quelle est la branche Git actuelle ? Quel commit correspond au code réellement ouvert ? Existe-t-il des modifications locales non intégrées ? Quels tests, fichiers de configuration, bases de données, outils de démarrage et artefacts sont présents ? Une copie ressemblant au bon dépôt ne suffit pas à prouver son identité.

L'investigation initiale peut être en lecture seule. Le but est de découvrir l'état réel, pas d'imposer immédiatement le modèle d'un projet fictif. Si l'application possède déjà un Product State ou un contrôleur canonique, le précontrôle de gouvernance existante protège cette autorité. Une seconde couche concurrente ne doit pas être installée pour faire disparaître un message de HOLD.

Un inventaire de fichiers et de hashes aide à vérifier la source, mais ne révèle pas à lui seul les exigences. Une table nommée `payments` ne précise pas les règles de remboursement. Un dossier `admin` ne prouve pas que l'accès est effectivement protégé. Le Planner vérifie les faits techniques avec la source et les règles fonctionnelles avec le CLIENT.

Protégez la confidentialité des éléments étudiés. Le dépôt public du framework n'est pas le lieu où placer des données privées du produit. Un manifeste peut référencer un artefact par identité ou hash sans copier son contenu vers un service tiers. La continuité ne requiert pas l'exposition de secrets.

3. Découverte : poser les questions qui comptent

Dans l'exemple d'un club, le Planner peut demander : qui travaille sur les inscriptions, quand un membre est-il considéré en règle, qu'arrive-t-il aux activités complètes, comment une annulation libère-t-elle une place et qui peut consulter l'historique des paiements ? Les réponses affectent les interfaces et les données.

Une question n'est utile que si sa réponse change une décision. Demander au CLIENT de choisir le nom d'une méthode alors que le problème est fonctionnel serait une perte de temps. À l'inverse, décider sans lui qu'un paiement ancien peut être supprimé serait franchir une frontière d'autorité.

Nature de la question	Exemple	Ce qu'elle détermine
Besoin	«Quelle tâche est pénible aujourd'hui ?»	La première capacité utile.
Règle métier	«Une dette bloque-t-elle l'inscription ?»	L'éligibilité et les messages.
Données	«Conserve-t-on les opérations annulées ?»	L'historique et le modèle de persistance.
Usage	«Deux employés travaillent-ils simultanément ?»	La concurrence et les accès.
Sécurité	«Qui voit les coordonnées personnelles ?»	Les autorisations et la visibilité.
Livraison	«Quand la première version doit-elle être utilisable ?»	Le périmètre et les dépendances critiques.

Le Planner confronte ces réponses aux décisions déjà présentes. Il ne repose pas

chaque question si la source canonique est claire. Il signale au CLIENT les choix qui restent ouverts et les conséquences d'une réponse. Si l'incertitude ne bloque pas une activité indépendante, cette activité peut être préparée sans inventer la règle manquante.

Les instructions de découverte et le cadre de spécification et livraison donnent des références. Elles ne remplacent pas la vérification des sources vivantes du projet concerné.

4. Baseline du produit et plan de livraison

Les réponses confirmées constituent un baseline de produit : objectifs, utilisateurs, comportements, critères de validation, exclusions et questions non résolues. En parallèle, l'ingénierie établit un baseline technique fondé sur le code : stack, dépendances, schéma, données actuelles, réseau, sécurité, tests, sauvegardes et déploiement prévu.

Les deux baselines se complètent. Une description métier ne prouve pas que l'application existe ; un schéma SQL ne prouve pas que le CLIENT l'approuve. Leur relation doit être explicite et pointée vers la version réelle de la source, avec owners et état courant vérifiables.

La livraison est découpée en capacités observables. Pour un club, on peut commencer par l'identité des membres, puis les cotisations, ensuite les activités et enfin les inscriptions. Chaque étape a des critères concrets et des dépendances. On évite d'autoriser des travaux qui s'appuient sur une règle de paiement non décidée.

Les jalons datés doivent être réalistes et conditionnés par des risques identifiés. «Créer cent commits» n'est pas un jalon de produit. «Un administrateur peut enregistrer un membre et le retrouver après redémarrage» fournit une preuve mesurable. Le Planner ajuste le plan lorsque les exigences, les ressources ou les contraintes changent réellement.

La documentation doit rester compacte et canonique. Le but n'est pas de recopier les mêmes exigences dans cinq fichiers `CURRENT`, mais de rendre les décisions et références accessibles, versionnées et non contradictoires.

5. L'adoption comme lien avec le projet réel

Adopter le framework signifie associer ses mécanismes de coordination au projet existant. Le Planner et l'ingénierie identifient source Git, Product State, responsabilités, données, artefacts, campagne actuelle et interfaces. Ils vérifient ce qui peut être transféré et ce qui ne doit pas être écrasé.

Le contrat de transfert et d'adoption décrit les enregistrements et les tests nécessaires. Une spécification d'adoption doit être examinée et validée avant toute promotion matérielle de l'état. Un exemple synthétique n'est pas un substitut à la source de vérité de l'application.

Pour un dépôt possédant déjà un contrôle canonique, un attachment de lecture et une éventuelle migration autorisée sont nécessaires. Une installation générique par-dessus une campagne active ou un Product State v1.2/v1.3 peut détruire l'historique d'autorité. Le scaffold v1.4 n'est pas un mécanisme d'upgrade automatique universel.

Les données persistantes nécessitent une preuve cohérente. Avec SQLite/WAL, un

hash du fichier principal peut ne pas représenter les transactions les plus récentes. Il faut un snapshot transactionnellement cohérent lorsque le contrat l'exige. Si la correspondance avec le code ou l'état courant est inconnue, on garde un HOLD sur les écritures dépendantes.

Les artefacts suivent le même principe : un fichier nommé `resultat-final.pdf` ne prouve pas son identité. Le chemin, le hash, les sources et le contexte doivent être reliés à la capacité lorsque cela compte pour la revue, la réutilisation ou la publication. Il n'est pas nécessaire de dupliquer des gigaoctets pour chaque agent ; on peut référencer l'artefact canonique.

6. L'épreuve du contexte frais

L'adoption doit être compréhensible par une nouvelle session qui ne connaît pas les discussions antérieures. Cette session retrouve le produit, les décisions actuelles, le propriétaire de la campagne, le code, la DB, les preuves et la prochaine action autorisée. C'est une épreuve plus pertinente que la simple présence de fichiers bien formés.

L'outil installé `reconstruct_context.py` vérifie certaines relations du contrôle portable. Il ne lit pas l'esprit du CLIENT et ne remplace pas l'inspection de l'application. Un nouveau Planner doit comparer ses résultats avec les sources et signaler les contradictions plutôt que de choisir le résumé le plus récent.

Si deux campagnes revendiquent la même ressource, elles doivent être réconciliées. Si un reviewer a approuvé un commit différent de celui qui est ouvert, la revue ne doit pas être réutilisée sans analyse. Si une DB n'a pas de témoin cohérent, on documente l'incertitude et l'owner capable de la résoudre.

Un HOLD précis peut indiquer : «La base utilise WAL et aucune photographie cohérente correspondant au code adopté n'a été trouvée ; les lectures sont autorisées, les écritures qui dépendent de cette cohérence sont suspendues». Ce diagnostic est plus utile qu'une déclaration vague d'impossibilité ou qu'un `READY` fabriqué.

7. Autoriser la première campagne

Après adoption, le Planner définit un charter de capacité : résultat visé, raison, limites, interfaces, dépendances, ownership, critères d'acceptation, gates et falsificateurs. Il ne transforme pas le charter en une liste de centaines de microtâches imposant chaque commande à I3.

Avant une autorisation importante, une première analyse stratégique reconstruit le produit final et les interactions entre données, runtime, sécurité, coûts, licence, récupération, utilisateur et étapes downstream. Une seconde analyse adversariale cherche des hypothèses fausses. Une nouvelle analyse stratégique supplémentaire n'est justifiée que par un changement de preuve ou un risque systémique matériel.

Une fois le charter approuvé, I3 possède l'exécution. Les tests échoués, refactorisations internes, nouvelles révisions techniques et reprises après crash restent normalement dans la même campagne. Le Planner reçoit un terminal consolidé quand il existe une véritable décision stratégique à prendre.

Cette frontière permet au CLIENT de rester responsable du produit sans devenir gestionnaire des erreurs internes. L'ingénierie peut travailler longtemps et s'adapter à des échecs sans perdre la continuité ni demander une nouvelle signature à chaque

correction.

8. Exemple de session avec le Planner

Le CLIENT explique : «Je veux une application locale pour gérer les membres d'une association ; nous utilisons actuellement une feuille de calcul et nous avons besoin d'une première version dans un mois». Le Planner demande quels parcours doivent fonctionner au premier jalon, qui utilisera le logiciel et si les données actuelles doivent être migrées.

Il vérifie le dépôt existant et les éventuels fichiers de données. S'il existe déjà une interface et une DB, il ne les jette pas pour reproduire un exemple. S'il n'y a pas de code, il propose une technologie proportionnée aux contraintes et explique les impacts importants sur l'installation et la maintenance.

La première capacité pourrait être l'enregistrement et la recherche de membres avec persistance. Les cotisations seraient un jalon suivant, une fois l'identité des membres stable. Les exclusions de la première version peuvent être les paiements en ligne, les notifications et les applications mobiles.

À la fin, le CLIENT sait quelles décisions ont été confirmées, quelles questions restent ouvertes, quelle démonstration aura lieu et ce qu'il doit encore choisir. Il n'est pas nécessaire qu'il connaisse tous les fichiers de contrôle ni les paramètres internes d'une transaction.

9. Un baseline que le CLIENT peut confirmer

Un baseline simple pourrait dire : «Un administrateur peut créer, chercher, modifier et désactiver des membres. L'historique des cotisations doit rester disponible. Le programme fonctionne sur ordinateur local. La première livraison n'effectue aucun paiement en ligne». Ces phrases deviennent des exigences seulement après confirmation.

L'ingénierie fournit ensuite les faits vérifiables : commit et configuration adoptés, environnement d'exécution, DB, méthode de sauvegarde, tests de persistance et de permissions. Les inconnues sont identifiées ; on n'invente pas de licence, de capacité du serveur ou d'approbation du CLIENT pour compléter une grille.

Ce baseline permet d'évaluer le résultat. Une interface séduisante qui ne sauvegarde pas un membre ne satisfait pas la capacité. Une application qui envoie des demandes à un fournisseur de paiement non autorisé dépasse le scope. Les critères d'acceptation mettent le comportement du produit avant le volume du code.

Lorsqu'un retour d'utilisation révèle une nouvelle règle, le Planner enregistre la supersession et examine les dépendances. Il ne maintient pas simultanément deux versions contradictoires comme vérité actuelle pour le même sujet.

10. Délimiter une première capacité

Un charter peut viser «Gestion des membres avec persistance démontrable». Il décrit création, recherche, édition et désactivation sur données fictives ; conservation des historiques nécessaires ; permissions ; interfaces de l'application ; et gates correspondant à l'environnement réel.

Les tests peuvent couvrir données invalides, doublons, lecture après écriture, redémarrage, accès non autorisé et parcours de l'interface. Une revue indépendante examine le candidat intégré exact. Le CLIENT intervient dans les jugements de produit et d'expérience, pas dans chaque décision de code.

Si désactiver un membre efface l'historique malgré l'exigence, I3 corrige dans la même campagne. Si le CLIENT demande ensuite un export fiscal, le Planner doit examiner les besoins, la confidentialité et les autorisations nouvelles. Les deux événements n'ont pas la même route.

Le terminal de I3 indique le candidat, les tests réalisés, les preuves physiques, les résultats de revue et les limites. Le Planner audite et accepte ou renvoie les falsificateurs pertinents. La capacité suivante dépendra de ce qui a été réellement accepté, pas d'une promesse contenue dans un rapport.

Suite : chapitre 03, fonctionnement quotidien et continuité de l'ingénierie.

Sources et portée

- Session de découverte du produit
- Exigences, stack et livraison
- Transfert et adoption
- Précontrôle des dépôts gouvernés
- Standard des charters de campagne
- Limites d'autorité du CLIENT

Les exemples ne constituent pas des spécifications en vigueur tant que les sources et le propriétaire du produit réel ne les ont pas confirmés.

03 — Fonctionnement quotidien

> Une session commence par les références **CURRENT**, le code et les preuves, non par le souvenir du dernier échange. I3 possède l’exécution technique dans les limites du charter ; les autorisations, la confidentialité, les coûts et les gates restent valables durant toute la campagne.

Dans ce chapitre

1. Reprendre le travail sans repartir de zéro
2. Transformer une demande en capacité vérifiable
3. Responsabilité continue du Controller
4. Autorisations et effets externes
5. Lire un rapport d’exécution de manière critique
6. Git, changements locaux et continuité
7. Ressources, artefacts et espace disque
8. Coordination, concurrence et rôle du CLIENT
9. Revue indépendante du candidat exact
10. Une routine quotidienne simple

1. Reprendre le travail sans repartir de zéro

Ouvrir un nouveau chat n’ouvre pas un nouveau projet. Le Planner ou le Controller récupère l’intention confirmée, le Product State, la branche et le commit réels, les éventuelles modifications locales, les campagnes actives, les owners, les témoins de DB et les artefacts. Les résumés de conversation peuvent indiquer une piste, mais ne doivent pas remplacer ces sources.

La reconstruction commence habituellement en lecture seule. Si l’état actuel n’est pas vérifiable, il vaut mieux déclarer l’incertitude précise que choisir une version arbitraire. Deux branches portant des noms similaires ou deux fichiers `final` peuvent contenir des résultats différents. Les références et hashes permettent de connaître le candidat qui a réellement été testé.

Une nouvelle session doit pouvoir répondre : quelle capacité est en cours, qui la contrôle, quel travail reste autorisé, quels gates ont été exercés et quel prochain effet peut être exécuté ? Si la réponse dépend uniquement de la mémoire d’un agent précédent, la continuité est encore insuffisante.

Le bootstrap du dépôt et le protocole de reconstruction de currentness décrivent le rôle des sources et des références. Un rapport `READY` n’est pas une preuve s’il pointe vers des éléments manquants ou contradictoires.

2. Transformer une demande en capacité vérifiable

Le CLIENT demande par exemple : «Je veux voir toutes les cotisations en retard et la date du dernier paiement». Le Planner vérifie la définition du retard, le calendrier,

les paiements partiels, les droits d'accès et les sources de données disponibles. Il distingue ce qui est nouveau de ce qui fait déjà partie d'une campagne approuvée.

Un charter de capacité indique le résultat à obtenir, les limites, les interfaces, les dépendances, les propriétaires, les critères d'acceptation et les falsificateurs. Il ne doit pas imposer chaque fonction, fichier et commit. Une telle microgestion réduirait la responsabilité opérationnelle du Controller et ferait remonter de simples défauts au niveau stratégique.

Le Planner peut exclure les fonctionnalités qui ne servent pas le premier résultat. Afficher les retards ne signifie pas automatiquement introduire des paiements en ligne, une notification par email ou un tableau de bord complexe. Ces effets nécessitent des décisions de produit distinctes.

Un bon objectif est observable. «L'administrateur peut rechercher les cotisations impayées pour une période donnée sans perdre l'historique» permet de concevoir des tests. «Rendre le backend plus professionnel» est trop vague tant qu'on n'a pas identifié une défaillance ou une capacité concrète à améliorer.

3. Responsabilité continue du Controller

Après autorisation, I3 planifie les tâches tactiques, implémente, exécute les tests, corrige les défauts, adapte des composants internes et prépare les candidats successifs. Le même charter peut durer plusieurs sessions et survivre à des interruptions. Un test échoué ou un problème de syntaxe ne constitue pas une invalidation stratégique.

Si la revue indépendante trouve que les données sont perdues après un redémarrage, I3 investigue le stockage, corrige la cause, ajoute une régression, reteste et soumet un nouveau candidat. Il ne crée pas une nouvelle campagne à chaque échec. Le Planner audite un terminal consolidé lorsqu'une décision stratégique réelle est possible.

Après deux retours ou plus d'une même catégorie de problème, le Controller doit rechercher la cause systémique. Une suite de petites corrections qui répare un cas tout en laissant les autres cassés n'est pas un progrès fiable. Le durcissement peut comprendre contrats, gestion des transactions, concurrence, tests négatifs et revue indépendante.

I4 peut aider sur une tâche bien délimitée, mais multiplier les agents n'améliore pas automatiquement la qualité. Les travaux en parallèle doivent respecter l'ownership des fichiers, bases, schémas et artefacts. Deux acteurs qui écrivent au même endroit sans coordination peuvent provoquer plus de conflits que de gains.

4. Autorisations et effets externes

Les lectures, tests locaux et corrections techniques prévues par un charter peuvent généralement être effectués par I3 dans ses permissions. Les actions irréversibles ou externes sont différentes : publier un package, envoyer un email, dépenser, exposer des données privées, supprimer une base réelle ou modifier un service de production peut nécessiter une validation explicite.

Action	Traitement approprié
Lire les sources et diagnostiquer.	Travail technique en lecture seule.
Modifier du code dans le charter.	Autorité de I3, avec contrôles adaptés.

Action	Traitement approprié
Ajouter une dépendance.	Vérifier origine, permissions, sécurité et nécessité.
Déployer ou publier.	Confirmer le gate de release et l'autorisation.
Dépenser via un fournisseur externe.	Respecter budget et consentement explicite.
Copier un token ou une base privée dans le chat.	À éviter ; employer le mécanisme de secrets approuvé.
Constater un conflit de Product State.	HOLD dépendant et réconciliation des sources.

Un utilisateur ne devrait pas décider de chaque détail de SQL ou des options de retry. L'ingénierie doit cependant expliquer les conséquences d'une nouvelle politique de confidentialité, d'un accès public ou d'une dépense récurrente. La frontière entre produit et technique dépend de l'effet réel de la décision.

Le framework peut fournir des contrôleurs portables de référence, notamment `campaignctl.py`. Leur présence ne prouve pas qu'un fournisseur propriétaire ait authentifié un agent ou exécuté une action native. Une affirmation d'exécution réelle doit être corroborée par l'environnement ou le service concerné.

5. Lire un rapport d'exécution de manière critique

Un rapport utile identifie le candidat exact, l'environnement, les actions réellement effectuées, les entrées, les résultats observés, les tests, la revue et les limites. Il doit permettre au Planner d'auditer la capacité et au CLIENT de savoir ce qui est utilisable sans parcourir des milliers de lignes de logs.

Les gates restent différents : unit pour la logique isolée, integration pour les interfaces ensemble, DB/currentness pour la cohérence des données et références, semantic pour la règle métier, physical pour une exécution réelle, perceptual/human pour l'expérience jugée par une personne, release pour l'artefact publié et ses autorisations.

Une capture d'écran prouve qu'une interface a été affichée à un moment donné. Elle ne prouve pas que les données sont sauvegardées. Une réponse JSON conforme ne prouve pas qu'un utilisateur trouve la fonction. Une CI verte du starter ne certifie pas un déploiement d'application. Chaque PASS doit avoir une portée explicite.

Demandez donc : quelle version a été testée, dans quel environnement, avec quel scénario, quels résultats et quels gates restent ouverts ? Le rapport peut être court, mais ses liens doivent mener aux sources canoniques réellement examinées. Le nombre de commits ou de tokens consommés n'est pas une mesure de fonctionnalité livrée.

6. Git, changements locaux et continuité

Git suit les changements de fichiers, pas l'ensemble de l'autorité. Un commit n'identifie pas à lui seul la campagne, l'acteur autorisé ni la cohérence de la DB. Les documents de currentness et les enregistrements d'exécution sont nécessaires pour reconstruire les relations entre code, données, artefacts et revues.

Avant de synchroniser, inspectez les modifications locales. Une opération aveugle de nettoyage peut effacer des fichiers non intégrés ; une fusion mal préparée peut mélanger deux décisions incompatibles. Préservez les versions utiles et comparez la base commune avec les changements distants. Un nom de branche n'est pas une garantie d'identité du contenu.

Les checkpoints peuvent servir à récupérer une session interrompue sans représenter une acceptation. À l'inverse, une capacité acceptée doit pointer vers un candidat et des preuves dont l'identité reste stable. La progression s'observe par le résultat produit, les gates gagnés et les risques réduits, non par la fréquence des commits.

Les outils de delta du dépôt permettent de limiter l'inspection aux changements pertinents. Ils ne remplacent pas le jugement sur les interfaces et n'autorisent pas la suppression de sources inconnues. Une nouvelle session doit constater le ref exact au lieu de se fier au souvenir d'un précédent agent.

7. Ressources, artefacts et espace disque

Un projet peut accumuler des clones, worktrees, caches, captures, fichiers audio/vidéo et builds. L'espace disque est une ressource protégée. Conservez de préférence un checkout persistant canonique par dépôt. Lorsque l'isolement de travaux concurrents le justifie, utilisez des worktrees temporaires qui partagent les objets Git et nettoyez-les une fois leur rôle terminé.

Avant de générer un artefact volumineux, vérifiez s'il existe déjà une version utilisable dont la provenance est connue. Un reviewer peut consulter le même fichier canonique par chemin et hash sans en obtenir une copie personnelle supplémentaire. Les rendus et intermédiaires sont conservés lorsqu'ils ont une valeur de preuve ou de produit, pas simplement «au cas où».

Le protocole d'inventaire et de garbage collection et `tools/workspace_gc.py` encadrent la suppression. Commencez par un inventaire read-only et un dry-run. N'effacez ni données humaines inconnues, ni sources, ni artefacts acceptés, ni éléments canoniques nécessaires à la reconstruction.

Le coût concerne aussi les services d'IA, le temps de calcul et les ressources réseau. Une opération techniquement facile peut entraîner des dépenses ou une exposition de données. Le Controller doit respecter le budget et les conditions d'arrêt autorisés, sans convertir une permission de test local en permission de facturation.

8. Coordination, concurrence et rôle du CLIENT

Lorsque plusieurs chantiers existent, chacun a un propriétaire et un périmètre de ressources. Deux équipes peuvent avancer en parallèle si leurs modifications sont indépendantes. Deux acteurs qui changent la même DB, le même schema ou le même contrat nécessitent des garanties de coordination. Le nom du projet ne suffit pas à conclure que deux actions sont sûres en concurrence.

Le MASTER OWNER peut intervenir sur les conflits d'ownership, les politiques dupliquées, la mauvaise intégration entre chantiers ou les campagnes qui restent prisonnières de faux terminaux. Il ne doit pas devenir un deuxième Planner reprenant chaque décision. La correction doit cibler la cause du dysfonctionnement systémique.

Le CLIENT confirme des règles métier, des limites de coût, des effets externes et des jugements d'expérience. Il peut servir de relais manuel entre une conversation

ChatGPT et un exécuteur qui n’est pas directement accessible : recevoir un message complet, le transmettre et consulter le rapport. Ce rôle ne le transforme pas en responsable de la cohérence technique.

Les messages adressés à I3 doivent être suffisamment autonomes : objectif, références actuelles, contraintes, gates et conditions de clôture. Évitez les fragments de patch que le CLIENT devrait assembler. Le système existe pour diminuer sa charge de coordination, pas pour la déplacer dans un autre outil.

9. Revue indépendante du candidat exact

Avant un terminal, I3 intègre un candidat dont les sources et configurations sont identifiables. Une revue indépendante examine ce candidat précis. Si la source change ensuite de manière matérielle, les contrôles affectés doivent être renouvelés. Il n’est pas nécessaire de répéter des tests indépendants d’une partie inchangée lorsque cette indépendance est démontrable.

Le protocole de freeze et de revue aide à structurer cette relation. Il ne prouve pas automatiquement le comportement d’une application sur un téléphone, dans un navigateur ou dans un environnement de production : l’essai physique reste nécessaire quand il fait partie du produit.

Si le reviewer relève un bug couvert par le charter, I3 corrige et représente un candidat. Si le résultat révèle une décision métier nouvelle, le Planner consulte le CLIENT. Si les témoins de currentness sont insuffisants, le terminal doit déclarer cette limite au lieu de la masquer par des tests verts.

Le but de la revue n’est pas d’interdire indéfiniment une capacité pour des détails sans importance, mais de découvrir et résoudre les risques matériels. Le Planner doit pouvoir comparer critères, preuves et impact, puis prendre une décision stratégique défendable.

10. Une routine quotidienne simple

En début de session, reconstruisez les sources, l’objectif, l’autorité et les blocages. Pendant l’exécution, conservez les checkpoints et vérifiez les effets importants. Avant un terminal, intégrez le candidat, exécutez les gates pertinents, obtenez la revue et rédigez un rapport traçable.

En cas d’interruption, conservez journal et identité de l’opération ; le prochain exécuteur doit reprendre sans dupliquer un effet. Un bug ordinaire revient à I3 dans la même campagne. Une modification véritable du produit revient au Planner. Une contradiction matérielle non résolue donne un HOLD spécifique pour les travaux dépendants.

À la fin de la semaine, le CLIENT doit connaître les capacités réellement utilisables, les risques restants, les décisions à prendre et les prochaines démonstrations. Si la réponse n’est qu’une liste d’activités techniques, il reste à transformer ce travail en valeur produit.

Suite : chapitre 04, demander une modification sans perdre le périmètre ni créer des campagnes artificielles.

Sources et portée

- Bootstrap et dépôt courant
- Protocole de reconstruction
- Intégrité de revue et terminal
- Outil d’inspection du delta
- Inventaire et gestion des artefacts
- Validation exécutable

Le framework décrit des mécanismes et des limites ; son installation ne certifie pas qu’un hôte d’IA tiers a exécuté une opération réelle.

04 — Demander des changements et de nouvelles capacités

> Une demande utile explique **le comportement attendu du produit**, son contexte et la manière de constater le résultat. Le CLIENT n’a pas à prescrire l’implémentation. Le Planner confirme les décisions nouvelles ; I3 corrige les défauts et effectue les changements techniques déjà couverts par le charter.

Dans ce chapitre

1. Décrire le résultat recherché
2. Distinguer fonctionnalité nouvelle, bogue et refactorisation
3. Un format simple pour exprimer une demande
4. Priorités, coût et chemin critique
5. Changements sensibles concernant données et sécurité
6. Retour utilisateur et évaluation perceptuelle
7. Renvoyer les défauts au même Controller
8. Éviter les politiques et campagnes doublonnées
9. Exemples de demandes bien délimitées
10. Confirmer que la nouvelle version est actuelle

1. Décrire le résultat recherché

Le CLIENT peut simplement dire : «Je souhaite consulter les cotisations impayées par membre et voir son dernier paiement». Cette phrase décrit une action observable. Le Planner vérifie le sens de «impayé», les périodes, les droits d’accès et les dépendances avec le stockage existant avant d’autoriser l’ingénierie.

Une formulation comme «rends le backend meilleur» ne permet pas de savoir si une modification a répondu au problème. «La recherche de membres prend plusieurs secondes lorsque notre base dépasse cinq mille enregistrements ; je voudrais retrouver rapidement les résultats corrects» fournit un contexte mesurable. L’ingénierie pourra analyser les performances et choisir une solution appropriée.

Le propriétaire du produit n’a pas à choisir préalablement un index, un cache ou une bibliothèque. Il doit préciser l’expérience souhaitée et les limites de coût, de données et de qualité. Les moyens techniques relèvent du Controller dans le cadre confirmé, avec remontée des décisions qui changent réellement le produit.

Un exemple visuel ou une courte capture peut aider, à condition d’être associé à un candidat et de ne pas exposer d’informations privées. Le problème, le résultat attendu et la manière de l’éprouver importent davantage qu’une grande quantité de commentaires ou captures non contextualisés.

2. Distinguer fonctionnalité nouvelle, bogue et refactorisation

Un **bogue** signifie que le programme ne respecte pas une règle déjà confirmée. Une **fonctionnalité nouvelle** ajoute ou modifie un comportement de produit. Une **refactorisation** restructure l'implémentation sans changer le contrat fonctionnel. Les trois nécessitent des tests, mais pas la même décision stratégique.

Exemple : le charter prévoit que la désactivation d'un membre conserve l'historique de ses paiements. La fonction efface cet historique. Il s'agit d'un bogue dans la campagne actuelle, et I3 doit corriger la persistance et faire une régression. Le Planner n'ouvre pas une nouvelle campagne «réparer le bouton».

Si le CLIENT demande ensuite que des membres inactifs puissent être réactivés avec des droits particuliers, c'est une règle nouvelle. Le Planner doit confirmer qui peut réactiver, comment les paiements sont traités et quel impact cela a sur les interfaces. On ne peut pas inventer une réponse et appeler «bug fix» ce qui modifie le produit.

Une refactorisation de classe ou d'API interne peut rester dans le mandat de I3. Si elle change le contrat public ou impose une migration de données au-delà du périmètre, il faut examiner les conséquences et la validité du charter. Le nom donné à l'opération ne remplace pas l'analyse des effets.

Situation	Route normale
Comportement existant contraire à une exigence.	Correction et tests par I3 dans la campagne actuelle.
Nouvelle règle métier demandée par le CLIENT.	Confirmation, impact et autorisation par le Planner.
Réorganisation technique sans effet fonctionnel.	Exécution par I3 avec contrôles appropriés.
Conflit entre projets ou propriétaires canoniques.	Coordination stratégique, éventuellement I1.
Publication, dépense ou partage externe nouveau.	Autorisation spécifique du responsable habilité.

3. Un format simple pour exprimer une demande

La plupart des demandes peuvent contenir cinq informations : état actuel, résultat attendu, contexte d'utilisation, contraintes et preuve de réussite. Il n'est pas nécessaire de demander au CLIENT de remplir un contrat complexe. Le Planner peut structurer ensuite la capacité et ses dépendances.

Par exemple : «Dans la liste des membres actifs, un membre désactivé apparaît encore. Je souhaite qu'il soit exclu de cette liste, tout en conservant son historique visible aux administrateurs. La correction doit être vérifiée après redémarrage». Si cette règle était déjà confirmée, l'énoncé est une description de bogue exploitable.

Autre demande : «La trésorerie doit exporter chaque mois un CSV des montants encaissés, sans numéros de téléphone». C'est une capacité nouvelle. Le Planner confirme le calendrier, le format, les autorisations, les risques d'exposition et le caractère actuel des chiffres. I3 implémente l'export et les tests adaptés.

Une préférence de présentation est également légitime : «Sur mobile, le nombre de places restantes doit être visible avant de remplir le formulaire». La solution technique peut être discutée, mais le jugement final sur la facilité d'usage ne doit pas être déduit automatiquement du nombre de tests verts.

4. Priorités, coût et chemin critique

Déplacer une fonctionnalité à plus tard n'implique pas nécessairement de changer ses exigences. Modifier une règle d'éligibilité, en revanche, peut affecter plusieurs capacités et nécessite une supersession explicite. Le Planner distingue ainsi calendrier, portée et sémantique du produit.

Le chemin critique indique quelles capacités conditionnent les suivantes. Si une inscription dépend du statut des cotisations, il vaut mieux avoir une source fiable de ce statut avant d'implémenter les règles d'inscription. Les éléments réellement indépendants peuvent avancer en parallèle si les ressources et contrats ne créent pas d'interférence.

Pour une première livraison rapide, une petite capacité fonctionnelle vaut souvent mieux qu'une architecture techniquement ambitieuse dont l'usage principal n'est pas démontré. Un CRUD correctement persisté peut révéler des problèmes de modèle et d'interface avant l'ajout de fonctions secondaires. La réduction du périmètre ne doit toutefois pas supprimer les gates nécessaires pour le comportement promis.

Le coût comprend développement, calcul, stockage, services externes et maintenance. Le nombre d'agents ou de commits ne mesure pas ce bénéfice. Un nouveau mécanisme doit résoudre une catégorie de défaut ou débloquer une capacité réelle ; sinon, il ajoute de la complexité sans valeur.

Un changement qui entraîne une dépense, une publication ou une opération irréversible demande un preflight et l'autorité correspondante. Une autorisation pour tester localement ne permet pas d'envoyer des données privées à un fournisseur ni de déclencher une facturation nouvelle.

5. Changements sensibles concernant données et sécurité

Certaines demandes ont des conséquences importantes malgré une interface simple. «Fusionner des membres en double» peut altérer des transactions historiques. «Rendre le rapport accessible à tous» peut exposer des coordonnées personnelles. «Changer de base» peut invalider des sauvegardes ou des références de currentness.

Le Planner reconstruit les propriétaires, entrées/sorties, données actuelles, dépendances downstream, règles d'invalidation et risques de récupération. I3 prépare les migrations, transactions, cas négatifs, idempotence, tests de concurrence et rollback dans les limites autorisées. Les étapes ayant des effets sur des données réelles demandent une preuve cohérente du résultat.

Pour SQLite en mode WAL, un snapshot doit correspondre à l'état transactionnel utile ; le fichier principal isolé peut être périmé. Si la source des données ne peut être identifiée, un HOLD spécifique protège les actions dépendantes. Le fait qu'un schéma de contrôle soit valide ne certifie pas les données qu'il décrit.

Il faut également distinguer désactivation logique, anonymisation, suppression physique et politique de conservation. Le CLIENT décide de la règle métier dans les limites légales ; l'ingénierie détermine comment l'appliquer de manière vérifiable. Un script destructif ne doit pas cibler un répertoire large ni des données dont l'ownership est inconnu.

Les artefacts générés dépendent parfois de sources, licences et paramètres. Un rapport ou une vidéo produit à partir d'entrées qui ont changé peut perdre sa validité. Renommer un fichier en `final` n'établit pas son lineage et ne rend pas actuelles des

références anciennes.

6. Retour utilisateur et évaluation perceptuelle

Une interface peut passer tous ses tests automatiques et rester difficile à utiliser. L'évaluation humaine est un gate distinct. Un retour utile décrit la tâche et le problème : «Quand aucune place n'est disponible, le message n'explique pas comment rejoindre une liste d'attente» est plus actionnable que «l'écran est mauvais».

Le CLIENT peut préciser la hiérarchie visuelle, l'accessibilité et les préférences du public. L'ingénierie propose des options et peut mesurer des propriétés objectives, mais elle ne doit pas inventer une approbation esthétique. Les captures de démonstration doivent correspondre au candidat réel, dans un environnement pertinent et avec des données fictives.

Si le problème d'UX viole un critère déjà confirmé, c'est un défaut à corriger par I3. S'il révèle une règle nouvelle, le Planner doit la confirmer. Un changement de couleur mineur peut rester dans le périmètre d'implémentation selon le charter, alors qu'une nouvelle interaction de paiement demande davantage d'analyse.

N'envoyez pas des captures comportant des noms réels, numéros de téléphone, tokens ou documents privés dans une issue publique. Un exemple synthétique et un rapport expurgé peuvent reproduire le problème sans exposer les utilisateurs du produit.

7. Renvoyer les défauts au même Controller

Lorsqu'un terminal est rejeté pour des problèmes déjà couverts par le charter, le Planner renvoie un ensemble **consolidé de falsificateurs** : exigence, candidat, résultat observé, preuve et gate concerné. I3 analyse, corrige, réintègre et obtient une nouvelle revue applicable. La campagne existante reste responsable de l'exécution.

Il n'est pas nécessaire de créer un nouveau charter ou un PASS stratégique pour chaque erreur de test. Ce processus ferait perdre au Controller son rôle d'ingénieur responsable et obligerait le CLIENT à arbitrer des corrections techniques. Une fois autorisé, I3 possède le debugging et les retries normaux jusqu'au terminal réel.

Lorsque le même défaut revient plusieurs fois, I3 doit cesser les corrections superficielles et examiner le sous-système de manière exhaustive. Cela peut impliquer la persistance, la concurrence, les invariants, la qualité des tests et une revue indépendante. Le but n'est pas de produire une succession de rapports `PASS` étroits, mais de résoudre la cause de la régression.

Une modification de l'intention du CLIENT reste une décision de produit ; elle ne doit pas être cachée sous «correction de bug». Symétriquement, un défaut normal n'est pas une invalidation stratégique du charter. La frontière entre les deux protège la vitesse et la responsabilité.

8. Éviter les politiques et campagnes doublonnées

Chaque préoccupation doit avoir un propriétaire canonique et une surface d'état courant. Deux campagnes qui changent une même interface sans coordination peuvent produire des contrats contradictoires. Deux fichiers `CURRENT` prétendant définir les mêmes exigences risquent de désorienter la prochaine session.

Avant d'ajouter une politique, cherchez le contrat existant. Les décisions stables, l'état mutable, l'autorité de campagne et les journaux de runtime ont chacun leur rôle. Une copie supplémentaire d'un résumé de vérité peut être utile comme index, mais ne doit pas devenir une deuxième autorité active.

La réutilisation des artefacts nécessite également une provenance. Si un rapport approuvé est repris ailleurs, il faut savoir quelles entrées l'ont produit et ce qui pourrait invalider cette approbation. Le stockage et les fichiers temporaires doivent être gérés sans multiplier des copies lourdes inutilement.

Un MASTER OWNER intervient lorsque les causes sont systémiques : duplication des politiques, churn de Planner, mauvaise séparation d'ownership ou défaillances d'intégration. Il doit corriger l'organisation concernée au lieu de créer une nouvelle étape de contrôle pour chaque bogue.

9. Exemples de demandes bien délimitées

A — Cotisations en retard. «Afficher les membres qui n'ont pas payé le mois, en tenant compte des versements partiels». Le Planner confirme la définition du mois, du paiement et de la dette ; I3 utilise la DB actuelle et teste les cas négatifs et la sécurité des consultations.

B — Perte d'historique. «Après désactivation, les paiements disparaissent alors qu'ils devaient rester consultables». C'est un bogue dans le charter. I3 corrige la cause, montre le readback et obtient la revue nécessaire sur le nouveau candidat.

C — Export CSV. «La trésorerie veut un total mensuel sans coordonnées personnelles». C'est une capacité nouvelle. Il faut confirmer le format, la période, le rôle autorisé, la précision des montants et les risques propres aux fichiers CSV.

D — Interface mobile. «Le bouton d'annulation n'est pas accessible sur un écran étroit». Il faut vérifier le parcours sur le dispositif concerné et distinguer la correction d'un défaut de présentation d'une nouvelle règle de comportement.

E — Nouvelle DB partagée. «Nous voulons remplacer le stockage local par un serveur multiutilisateur». Cela change réseau, concurrence, authentification, sauvegardes, coût et récupération. Une analyse stratégique et une migration avec rollback sont nécessaires.

Ces exemples n'imposent pas un template rigide. Ils montrent comment séparer intention, décisions ouvertes, exécution et preuves, pour éviter que l'architecture ne détourne le temps des vrais besoins du produit.

10. Confirmer que la nouvelle version est actuelle

Après la modification, I3 doit identifier le candidat intégré, les tests exécutés, les résultats physiques et les artefacts affectés. Le Product State, le code, les données et la revue doivent correspondre à cette identité. Un diff ou une déclaration «fait» ne prouvent pas que l'utilisateur pourra accomplir la tâche.

Le Planner audite le terminal contre les exigences et enregistre acceptation, retour à I3 ou question de produit. Le CLIENT juge les critères réservés, notamment l'expérience. Les dépendants downstream ne doivent être débloqués qu'à partir d'une capacité effectivement acceptée.

Une session future doit pouvoir retrouver la décision sans relire le chat. Si elle découvre un lien cassé, une source périmée ou deux exigences actives contradictoires,

elle doit réconcilier cette situation. Une modification n’est durable que si son résultat et ses références restent vérifiables.

Le meilleur indicateur d’une demande bien traitée est la réduction d’incertitude : on sait ce qui a changé, pourquoi, qui a approuvé la règle, comment le résultat a été testé et quelle action devient possible ensuite.

Suite : chapitre 05, incidents, erreurs et récupération du projet.

Sources et portée

- Limites du CLIENT
- Standard des charters
- Cycle des capacités et preuves
- Découverte et plan de livraison
- Revue du candidat exact
- Support public v1.4

Les exemples fictifs ne créent pas des exigences réelles ni des autorisations de campagne. Le propriétaire et les sources du produit concerné restent déterminants.

05 — Erreurs, incidents et récupération

> Récupérer un projet ne signifie pas simplement faire disparaître un message d’erreur. Il faut revenir à un état **vrai, cohérent et vérifiable**, en préservant les sources, les propriétaires, les journaux de transaction et les preuves pertinentes. Un défaut ordinaire reste sous la responsabilité de I3 ; une contradiction matérielle de currentness ou d’autorité exige une autre réconciliation.

Dans ce chapitre

1. Conserver les preuves avant de réparer
2. Pannes d’environnement, de comptes et de permissions
3. Lorsque la validation de contrôle échoue
4. Git ne sert pas à effacer les problèmes
5. Défauts répétés et causes systémiques
6. Sécurité, confidentialité et droits externes
7. Prouver la fin de l’incident
8. Trois scénarios de récupération
9. Opérations sensibles et autorisations
10. Ce qu’un test vert ne prouve pas
11. Retours d’architecture et données locales

1. Conserver les preuves avant de réparer

Lorsqu’une opération échoue, sauvegardez le message exact, la commande ou l’action, la date, le dépôt et worktree, le commit ou candidat concerné, ainsi que les éventuels effets partiels. Ne supprimez pas immédiatement `.agentic-sdlc/`, les fichiers de DB ou les logs. Ces données peuvent être indispensables pour comprendre la cause et reprendre sans perdre du travail.

Commencez par classer l’incident. Une dépendance Python absente, un conflit Git, une règle métier mal implémentée, un journal transactionnel incomplet, une autorisation de publication manquante et une base de données incohérente sont des problèmes différents. Le propriétaire et la démarche ne sont pas interchangeables.

Signal	Première réponse sûre
Un test de la fonctionnalité échoue.	I3 conserve la reproduction et corrige dans le charter courant.
Un validateur rejette une référence canonique.	HOLD sur l’effet dépendant, inspection de la relation fautive.
Des changements Git locaux et distants divergent.	Préserver les deux et étudier les deltas.
Des données réelles peuvent être endommagées.	Suspendre les écritures sensibles, protéger backups, WAL et journaux.
Il manque le droit de publier ou de payer.	Ne pas effectuer l’effet externe.

Signal	Première réponse sûre
Un hôte a cessé de répondre pendant une opération.	Effet inconnu ; lire l'état avant toute nouvelle tentative.

La correction est terminée lorsqu'on a démontré le gate concerné sur le candidat courant. Supprimer une ligne de log ou modifier un champ `READY` peut produire un rapport agréable sans réparer le produit. Les investigations en lecture seule peuvent souvent continuer pendant que les écritures sensibles restent bloquées.

Le CLIENT doit connaître les conséquences de l'incident et les choix qui lui appartiennent. Il ne doit pas gérer chaque conflit de fichiers ou décider de l'algorithme de récupération. I3 assume les défauts techniques couverts par le charter et rend compte de manière vérifiable.

2. Pannes d'environnement, de comptes et de permissions

Python est introuvable. Vérifiez le chemin, l'interpréteur actif, sa version et le PATH. Installez les dépendances déclarées depuis une source fiable. Ne modifiez pas le Product State pour contourner un problème de configuration locale.

Impossible d'accéder à GitHub. Distinguez absence d'authentification, permissions insuffisantes, installation du connecteur, visibilité du dépôt et branche inaccessible. Un refus de lecture ne justifie pas de demander tous les droits du compte ni de copier un token en clair dans la conversation.

L'agent travaille dans le mauvais projet. Comparez dossier courant, remote, ref, fichiers et éventuels worktrees. Deux chemins similaires ne sont pas interchangeables. Avant d'écrire, il faut confirmer que le projet ouvert correspond réellement à celui que le CLIENT a autorisé.

La session ou le service expire. La fin d'un thread ne termine pas la campagne. Le nouvel acteur reconstruit Goal, charter, owner, candidate, checkpoint et effets pendants. Une réponse interrompue ne signifie pas nécessairement qu'une action externe n'a pas été exécutée.

Permission du système refusée. Identifiez la cible exacte et les droits requis. Ne contournez pas un contrôle de sécurité en utilisant un autre chemin ou une suppression plus large. Lorsque l'autorisation est réservée, interrompez l'effet et demandez la décision correspondante.

Les erreurs d'environnement se résolvent souvent simplement, mais le diagnostic doit précéder les mutations. Une réparation aveugle peut masquer une divergence de version ou effacer des données que le Controller aurait pu récupérer.

3. Lorsque la validation de contrôle échoue

Le `tools/validate_project.py` installé examine la structure et les contrats de `.agentic-sdlc/`. Il peut détecter un propriétaire dupliqué, un `CURRENT` incohérent, un candidat différent de celui revu ou une transition incomplète. Il ne prouve pas que l'application se connecte réellement à sa base, que les permissions métier fonctionnent ou que l'expérience utilisateur est satisfaisante.

Avant toute réparation, conservez le résultat, le root du contrôle, le commit actuel, les changements locaux et les témoins de DB. Identifiez si une relation est absente,

périmée, contradictoire ou bloquée par une transaction. Ne modifiez pas directement les valeurs d'état pour obtenir un rapport vert.

Les mises à jour de plusieurs fichiers utilisent un journal de récupération transactionnelle. Lorsque les hashes avant/après ne correspondent pas à l'effet attendu, la procédure doit préserver le conflit plutôt que de réécrire silencieusement les fichiers d'un autre acteur. Un journal pendant peut légitimement bloquer les opérations suivantes.

SQLite en mode Write-Ahead Logging ajoute une difficulté. Le fichier principal `.db` peut être en retard par rapport à l'état transactionnel courant. Un snapshot cohérent, incluant les effets pertinents du WAL, est nécessaire lorsque la currentness en dépend. Le précontrôle des projets gouvernés documente ces limites ; ce n'est pas un outil automatique de réparation de production.

Après la récupération du contrôle, vérifiez séparément le comportement physique de l'application. Un schéma JSON valide ne remplace pas une opération de lecture/écriture de DB et une preuve de DB ne remplace pas la vérification d'une interface.

4. Git ne sert pas à effacer les problèmes

Un conflit Git représente deux changements potentiellement valables. Ne choisissez pas mécaniquement «version locale» ou «version distante». Examinez la base commune, les différences, les owners et les décisions qui ont conduit aux modifications. Le but est d'intégrer le bon comportement, pas seulement de rendre le statut Git propre.

N'utilisez pas de reset destructif ou de suppression d'un dépôt entier pour éviter la réconciliation. Les modifications non intégrées peuvent appartenir à l'utilisateur ou à une campagne active. Préservez-les tant que leur provenance et leur pertinence n'ont pas été établies.

Les outils de différentiel de source et d'inventaire/gestion des artefacts aident à comprendre l'espace de travail. Ils ne permettent pas d'effacer arbitrairement du code, des données inconnues, des assets acceptés ou des éléments de preuve canoniques. Une opération de nettoyage commence par un inventaire et un dry-run.

Les commits ne sont pas des certificats d'identité d'acteur. La preuve qu'un agent avait l'autorité pour écrire doit provenir des surfaces de contrôle pertinentes, pas seulement d'un nom d'auteur Git. Une branche qui avance ne prouve pas non plus que les revues ou les témoins de DB correspondent au nouveau candidat.

Lorsqu'un fichier change après une revue favorable, les contrôles concernés doivent être renouvelés. La récupération ne doit jamais transformer une approbation d'ancienne version en acceptation implicite d'une autre source.

5. Défauts répétés et causes systémiques

Un défaut déjà couvert par le charter appartient au Controller. Tests échoués, migration en périmètre, refactorisation interne, bug découvert par un réviseur ou interruption du runtime ne nécessitent pas une nouvelle campagne ni une nouvelle analyse du Planner.

Si le même type de panne réapparaît dans plusieurs candidats, le Controller doit arrêter les microcorrections successives. Il faut examiner le sous-système :

transactions, contrats, concurrence, données, tests négatifs, limites d'intégration et causes de la régression. Un durcissement complet avec revue indépendante peut être nécessaire.

Prenons une désactivation de membre qui supprime ses paiements. Corriger un seul cas sans tester les paiements partiels ou les interactions avec d'autres tables peut laisser le problème vivant. Une solution causale examine les relations de persistance et démontre la conservation du bon historique après un redémarrage.

Le Planner revient lorsque I3 présente un terminal stratégique ou lorsqu'une preuve nouvelle invalide réellement le charter. La difficulté du bug n'est pas, à elle seule, une modification de la stratégie. Cette frontière protège le CLIENT de cycles de planification sans résultat produit.

À l'échelle d'un portefeuille, le MASTER OWNER doit aussi détecter les défauts systémiques : polices dupliquées, Controllers qui n'assument pas les corrections, confusion des responsabilités et problèmes répétés d'intégration entre chantiers. La solution doit viser le mécanisme responsable, pas ajouter une nouvelle cérémonie à chaque incident.

6. Sécurité, confidentialité et droits externes

Les logs et rapports publics doivent être expurgés. N'insérez pas de tokens, mots de passe, données de membres, chemins privés, bases complètes ou transcriptions de conversations dans des issues accessibles à tous. Une fixture fictive peut souvent reproduire le défaut sans exposer les personnes concernées.

Le starter n'autorise pas l'envoi du code vers un fournisseur externe. Les modèles et connecteurs restent facultatifs et nécessitent une configuration et des permissions spécifiques. Une solution qui exige facturation, publication ou transmission de données doit obtenir l'autorisation avant de réaliser l'effet.

Une licence libre du framework ne couvre pas automatiquement des ressources tierces. Images, polices, musique et datasets ont leurs propres droits. Évaluez-les de manière proportionnée à l'usage réel, sans présumer qu'une permission existe ni imposer un contrôle documentaire sans lien avec le risque.

Une sauvegarde de production peut contenir des informations sensibles. La rendre publique pour simplifier un test n'est pas acceptable. Les essais doivent protéger confidentialité, intégrité et accès. L'effacement, la restauration et la modification de données sont des actions distinctes qui demandent des contrôles différents.

7. Prouver la fin de l'incident

Une réparation terminée doit pouvoir être reproduite. Le rapport indique l'incident, la source et le candidat, la cause, la correction, les tests exécutés, l'état persistant, la revue et les limites restantes. Le message d'erreur ne doit pas simplement disparaître ; le comportement attendu doit fonctionner dans l'environnement pertinent.

Pour une migration, vérifiez l'état après écriture et, si nécessaire, après redémarrage. Pour une interface, ouvrez réellement l'écran avec des données appropriées. Pour un effet externe interrompu, lisez l'état distant avant de réessayer afin d'éviter un doublon. La récupération doit respecter idempotence et autorisation.

Si la correction modifie le candidat après la revue, renouvelez les gates affectés. I3 peut produire un terminal technique avec des preuves, mais ne peut pas accorder de

lui-même le consentement du CLIENT à une publication ou à une qualité subjective.

Un compte rendu de récupération doit être court mais reconstructible. Un autre ingénieur doit pouvoir identifier les fichiers, l'environnement et la prochaine action sans lire la conversation entière.

8. Trois scénarios de récupération

Scénario A — Installation interrompue. Le scaffold a écrit certains fichiers avant la fermeture du terminal. Inspectez la destination, gardez les données existantes et comparez l'état avec le contrat. Utilisez la route de récupération sûre ; n'effacez pas toute l'application pour relancer l'installateur.

Scénario B — Paiement au résultat inconnu. Une opération reçoit un timeout après l'envoi. Le Controller doit déterminer si un paiement a réellement été enregistré en consultant un identifiant d'opération ou un état transactionnel. Répéter aveuglément peut provoquer un double effet. Si la preuve manque, gardez un HOLD sur la répétition dangereuse.

Scénario C — Revue périmée. Le réviseur a approuvé A, mais le code a été modifié pour B. La validation de A n'accepte pas B. I3 identifie les différences, répète les contrôles concernés et obtient une revue du candidat intégré exact. Les preuves non affectées peuvent être conservées si leur indépendance est justifiée.

Ces scénarios ont des causes différentes : effet partiel, idempotence et identité du candidat. Aucun ne nécessite automatiquement une campagne stratégique nouvelle. Chacun exige une preuve cohérente et un propriétaire opérationnel clair.

9. Opérations sensibles et autorisations

Avant une opération à risque, déterminez le système cible, le propriétaire de la ressource, les conséquences, la réversibilité, les données exposées, les moyens de retour arrière et les permissions réelles. Une action automatisable n'est pas nécessairement autorisée.

Opération	Limite à respecter
Inspecter les sources et réaliser des tests locaux.	Permissions techniques et périmètre courant.
Modifier du code sous charter.	Ownership, tests et revue de I3.
Changer des données ou un schéma de production.	Preflight, migration, sauvegarde et preuve de readback.
Supprimer des fichiers ou bases.	Cible exacte et autorisation matérielle si nécessaire.
Envoyer des données à des tiers.	Confidentialité, licence et consentement spécifique.
Publier un artefact ou une release.	Gate de distribution et autorité adéquate.

Si une autorisation manque, arrêtez l'effet et demandez-la. Ne cherchez pas une voie détournée pour contourner le refus. Si la difficulté se limite à un problème technique déjà autorisé, I3 le résout sans transférer au CLIENT les décisions internes d'ingénierie.

La récupération automatique suit les mêmes principes. Un script de nettoyage ne doit pas supprimer un répertoire large dont l'ownership est inconnu. Un retry doit

être sûr quant à ses effets. La vitesse ne justifie pas de perdre la preuve de ce qui a été modifié.

10. Ce qu'un test vert ne prouve pas

La validation de structure du starter vérifie des contrats documentaires et des références. Un test de scaffold vérifie l'installation en environnement contrôlé. Un scénario synthétique de récupération prouve un comportement limité du mécanisme testé. Aucun de ces tests ne certifie automatiquement un produit tiers en production.

Une fonction unitaire peut réussir tandis que l'API est mal connectée. L'intégration peut passer avec une DB qui ne correspond pas à la source actuelle. Une interface peut s'ouvrir et rester inutilisable pour son public. Les gates de DB/currentness, semantic, physical et perceptual doivent être distingués.

Une publication exige aussi de vérifier l'artefact exact, sa version, son checksum et son accessibilité sur le service visé. Un ZIP construit localement et une tag Git ne prouvent pas que tous les assets d'une release ont réellement été publiés. L'acceptation doit nommer le gate et sa portée.

Quand un gate n'a pas de sens pour la capacité, expliquez pourquoi. Quand il est requis et non exécuté, ne le remplacez pas par un test moins pertinent qui a déjà réussi. Le but est de vérifier les risques réels sans recherche interminable de défauts accessoires.

11. Retours d'architecture et données locales

En v1.4, un générateur local de rapports restreints de métadonnées peut être actif par défaut. Un workflow mensuel, lorsqu'il est installé, enregistre les résultats dans les propres artefacts GitHub Actions du projet. La distribution publique ne contient pas d'endpoint de collecte externe.

L'option `--no-feedback-workflow` désactive l'ordonnancement, pas le générateur local. Pour arrêter tous les rapports, réglez `enabled=false` dans `.agentic-sdlc/feedback/ARCHITECTURE_FEEDBACK_CONFIG.json`. Consultez la politique de confidentialité et le workflow exact avant d'autoriser toute transmission.

Un rapport de bogue du framework peut indiquer la version, la plateforme, les étapes de reproduction, des données synthétiques et des messages expurgés. Il n'a pas besoin du code privé ni des conversations de l'utilisateur. Les informations nécessaires à la correction sont différentes de celles qui permettraient d'identifier une personne.

La récupération réussie se reconnaît au fait qu'un nouveau travailleur peut identifier l'état courant, le résultat observé, les limites et la prochaine action. Si seule l'ancienne conversation donne ces réponses, l'état durable reste incomplet.

Suite : chapitre 06, clôture des capacités et acceptation avec des preuves exactes.

Sources et portée

- Récupération des transactions
- Précontrôle des projets déjà gouvernés
- Outil d'inspection de source
- Inventaire et suppression sûre

- Confidentialité et feedback
- Support public v1.4

Les mécanismes précis de récupération dépendent du projet et des permissions. Aucun exemple de ce chapitre n’autorise la destruction d’un état réel.

06 — Clôture des capacités et acceptation

> Une capacité se clôt sur **un candidat exact et les preuves qui lui correspondent**. Un terminal technique de I3 permet l’audit stratégique ; il n’approuve pas automatiquement le produit, ne remplace pas une évaluation humaine et ne donne pas le droit de publier ou de déployer. La clôture distingue ces décisions tout en évitant de créer une campagne pour chaque bogue.

Dans ce chapitre

1. Une capacité est un résultat de produit
2. Les cinq composantes d’une clôture
3. Le candidat exact est l’unité de revue
4. Preuves insuffisantes sans campagnes artificielles
5. Accepter, reprendre ou changer d’objectif
6. Revues périodiques et règle d’arrêt
7. Un enregistrement de clôture vérifiable
8. Exemple de rejet et de correction
9. Produit utilisable contre framework vert
10. Que faire après la clôture

1. Une capacité est un résultat de produit

«Gestion des membres» n’est pas terminée parce qu’un formulaire existe ou que des commits ont été créés. Elle est terminée lorsque l’utilisateur prévu peut effectuer les actions convenues dans l’environnement pertinent, que les données sont conservées conformément aux exigences, que les contrôles applicables passent et que les responsables compétents acceptent les preuves.

Le Planner définit le charter et les limites de la capacité. I3 développe, intègre un candidat identifiable, exécute les tests et organise la revue indépendante pertinente. Le Planner audite le terminal à partir des exigences actuelles. Le CLIENT conserve son autorité sur les choix de produit, l’expérience et les effets externes réservés.

Une campagne peut comprendre plusieurs sessions, reprises après crash et candidats successifs. Elle ne se termine pas parce qu’un agent atteint une limite de contexte. Les interruptions ne révoquent pas les décisions précédentes, mais ne créent pas non plus de nouvelles autorisations. Le résultat doit rester reconstructible sans mémoire du chat.

Une capture d’écran ne suffit pas à prouver une fonctionnalité qui modifie des données. Un backend qui répond à un test unitaire ne démontre pas qu’un utilisateur peut accomplir la tâche. Les preuves doivent correspondre au résultat réellement promis.

2. Les cinq composantes d’une clôture

Composante	Question à résoudre
Résultat visible	L'utilisateur peut-il effectuer la tâche dans l'environnement prévu ?
Vérifications applicables	Les contrôles de logique, intégration, données, sécurité et expérience pertinents ont-ils été exercés ?
État courant cohérent	Code, configuration, exigences, DB et artefacts correspondent-ils au candidat ?
Absence de blocage matériel dissimulé	Un défaut ouvert invalide-t-il le résultat annoncé ?
Acceptation compétente	Planner et CLIENT ont-ils examiné les preuves relevant de leurs responsabilités ?

Tous les projets n'exigent pas la même profondeur de contrôle. Un script interne simple n'a peut-être pas de gate perceptuel significatif. Une application qui conserve des cotisations exige des vérifications de persistance, de droits d'accès et de parcours utilisateur. Un contrôle ne peut pas remplacer silencieusement un autre qui importe pour le produit.

Les gates à distinguer sont :

- **Unit :** logique isolée conforme aux entrées et cas négatifs attendus.
- **Integration :** composants interconnectés respectant leurs interfaces réelles.
- **DB/currentness :** données, migrations, états et références actuelles cohérents et durables.
- **Semantic/generalization :** comportement conforme à la règle métier, au-delà d'une fixture étroite.
- **Physical :** application ou artefact exécuté réellement dans l'environnement prévu.
- **Perceptual/human :** expérience et qualité subjective évaluées par une personne compétente lorsque c'est nécessaire.
- **Release :** distribution exacte, permissions, droits, checksums et conditions externes vérifiés.

Une validation JSON ne prouve pas les opérations CRUD. Un screenshot ne prouve pas la DB. Une CI verte ne prouve pas l'expérience. Chaque gate a un candidat, un propriétaire, des critères et des éléments de preuve. Des contrôles disproportionnés peuvent retarder une livraison sans réduire les risques ; des contrôles manquants peuvent produire une fausse acceptation.

3. Le candidat exact est l'unité de revue

Le réviseur indépendant examine le même candidat que celui proposé à l'acceptation. Selon le projet, cela comprend le commit, la configuration, la version du schéma, les témoins de base, les hashes d'assets et l'environnement. Une revue du candidat A n'autorise pas une version B modifiée, même si le nom de la branche ou du fichier est identique.

Le parcours normal consiste à intégrer, figer l'identité pertinente, réaliser les tests physiques et la revue, puis présenter le terminal au Planner. Une modification matérielle postérieure exige de renouveler les contrôles affectés. Des preuves

réellement indépendantes des changements peuvent être conservées lorsque leurs dépendances sont connues.

Le protocole de freeze, revue et terminal définit cette relation. Il ne permet pas au contrôleur de référence d'imiter l'acceptation d'un utilisateur réel ou le comportement d'un système externe qu'il n'a pas exécuté.

Après une revue technique positive, les gates humains, de sécurité, de droit ou de publication peuvent rester ouverts. Le rapport doit le dire clairement. Un terminal signifie «prêt pour le jugement suivant», pas «toutes les autorisations sont acquises».

Cette exigence de précision protège aussi contre des faux négatifs. Si une revue ancienne concerne une fonctionnalité inchangée, elle peut rester pertinente. Mais il faut pouvoir démontrer la relation de dépendance plutôt que supposer que l'ancien PASS est universel.

4. Preuves insuffisantes sans campagnes artificielles

Si le terminal ne satisfait pas une condition déjà comprise dans le charter, le Planner renvoie à I3 un ensemble **consolidé de falsificateurs** : exigence, candidat, preuve, résultat observé et gate affecté. I3 corrige, teste, intègre et soumet un nouveau candidat dans la **même campagne**.

Un bug, un test d'intégration échoué, un retry, une refactorisation interne ou un avis de reviewer concernant une erreur de portée sont des sujets d'exécution. Il n'est pas justifié de créer un nouveau charter ou des PASS stratégiques supplémentaires à chaque défaut.

Lorsque le même type de défaillance revient plusieurs fois, I3 doit durcir le sous-système de manière approfondie. Le travail peut demander une inspection des transactions, de la concurrence, des tests négatifs, des invariants et de la revue indépendante. La priorité est d'éliminer la classe de défauts, non de produire une série de faux terminaux.

Une décision de produit nouvelle, une autorisation modifiée ou une incompatibilité stratégique peuvent exiger de revoir le charter. La difficulté d'un bogue, à elle seule, ne modifie pas le périmètre. Cette distinction empêche les cycles inutiles de Planner et les extensions de portée non autorisées.

Un bon terminal est concis : identité et environnement du candidat, parcours réellement exercé, preuves des gates réussis, gates ouverts et leur impact, risques nouveaux et propriétaire de la prochaine action. Ce document indexe des preuves canoniques et ne constitue pas un deuxième Product State.

5. Accepter, reprendre ou changer d'objectif

Accepter. Lorsque les preuves appropriées existent, le Planner enregistre l'acceptation et met à jour les références canoniques. Le CLIENT effectue l'acceptation réservée sur le produit et l'expérience. Les travaux downstream ne doivent dépendre que des capacités réellement acceptées.

Reprendre après interruption. Un nouvel exécuteur reconstruit Goal, charter, checkpoint, source, DB et effets pendants. Il ne déclare pas le travail terminé parce que l'ancienne conversation est inaccessible. Le résultat durable a plus d'autorité qu'un souvenir ou qu'un résumé rassurant.

Changer de priorité. Un report de livraison ne modifie pas forcément les exigences. Une nouvelle politique de cotisation, en revanche, modifie le produit. Les supersessions et conséquences doivent être enregistrées ; on ne conserve pas deux règles contradictoires comme état courant du même sujet.

Conserver HOLD. Si la correspondance entre le candidat et l'état actuel ne peut pas être vérifiée, les acceptations dépendantes restent bloquées. Les analyses sûres peuvent continuer. HOLD n'est ni un échec définitif ni une permission d'inventer les faits manquants.

6. Revues périodiques et règle d'arrêt

Une revue hebdomadaire devrait permettre au CLIENT de demander : quelles capacités sont acceptées, que peut-on démontrer, quel risque de délai est réel, quel budget reste justifié, quelle décision dépend de moi et quelle est la prochaine livraison vérifiable ? Le rapport s'appuie sur le produit, pas seulement sur les activités.

Le nombre de commits, de documents, d'agents, de tests ou de tokens n'est pas une mesure de progression produit. Ces opérations peuvent être nécessaires, mais une fonctionnalité qui ne fonctionne pas ne devient pas plus utile parce qu'elle a généré beaucoup de rapports.

Le starter comporte une règle d'arrêt : une fois les gates de release satisfaits, il ne faut pas accumuler des politiques génériques sans défaillance concrète ou capacité manquante. Chaque mécanisme supplémentaire a un coût de maintenance et peut introduire de nouvelles contradictions.

Le même principe vaut pour une application. Accepter trop tôt un résultat cassé est dangereux ; chercher indéfiniment des microdéfauts sans impact peut empêcher toute livraison. Le Planner et le CLIENT doivent pouvoir distinguer les risques matériels des améliorations secondaires.

7. Un enregistrement de clôture vérifiable

Une personne qui n'a pas participé à la campagne doit pouvoir retrouver les éléments essentiels sans relire le chat. L'enregistrement peut pointer vers les surfaces canoniques :

Champ	Preuve recherchée
Candidat	Identité précise du code, de la configuration, de l'état et des artefacts.
Résultat	Entrée, action et sortie observée du parcours de bout en bout.
Tests	Résultats, environnement et commandes pertinents.
Données persistantes	Relecture après écriture, redémarrage et récupération si nécessaires.
Sécurité et droits	Contrôles proportionnés à la confidentialité et aux ressources externes.
Revue indépendante	Réviseur, périmètre et candidat effectivement examinés.
Changements après revue	Différences et contrôles renouvelés.

Champ	Preuve recherchée
Acceptation humaine	Décision authentique, jamais déduite d'une capture.
Route suivante	Accepter, renvoyer à I3, demander une décision ou HOLD motivé.

Ce tableau ne doit pas être recopié dans un second fichier `CURRENT` concurrent. C'est une carte vers les sources existantes. Un lien vers un artefact manquant ou une fixture sans rapport avec le produit est insuffisant pour accepter le résultat.

La preuve de release du framework et la preuve de fonctionnalité d'une application doivent rester distinctes. Le starter peut avoir passé des validations internes et des essais d'installation portables sans démontrer l'ergonomie ou la logique métier d'un produit externe.

8. Exemple de rejet et de correction

Supposons qu'une application permet de créer et désactiver des membres. Un reviewer découvre que la désactivation supprime leurs paiements historiques, alors que le charter exige leur conservation. Le Planner renvoie un falsificateur précis : «La désactivation d'un membre avec paiements doit préserver les transactions après redémarrage ; le candidat X les perd».

I3 recherche la cause de persistance, corrige les relations, ajoute une régression pertinente, démontre la lecture après redémarrage, intègre un nouveau candidat et demande une revue indépendante. Il n'ouvre pas une campagne «réparer désactivation» et n'oblige pas le CLIENT à choisir les contraintes de clés étrangères.

Si le reviewer avait examiné A et que la correction produit B, l'ancien PASS ne suffit pas pour B. Les tests non affectés peuvent rester réutilisables, mais le nouveau comportement et les données modifiées doivent être testés et revus sur la bonne identité.

À l'inverse, si le CLIENT demande la réactivation d'un membre avec des permissions spéciales, cela ajoute une règle. Le Planner confirme les conditions et l'impact stratégique. Une nouvelle fonctionnalité n'est pas un simple bogue de l'ancien système.

9. Produit utilisable contre framework vert

Un contrôleur peut valider une campagne synthétique et atteindre un terminal sans qu'une interface de produit existe. Pour accepter une gestion des membres, il faut ouvrir l'application, créer une fiche fictive, la retrouver, la modifier, la désactiver et conserver l'historique. Si la persistance est importante, redémarrer et relire. Si des permissions existent, tester également l'accès refusé.

Le `tools/verify_v1_release.py` vérifie une version précise du framework avec des fixtures. Source, installation portable Linux/Windows, distribution publique et capacité d'une application construite avec ce framework sont des gates distincts. La matrice de support v1.4 précise les comportements réellement exercés.

Dire «tous les tests sont passés» sans nommer le candidat et les tests peut induire le CLIENT en erreur. Il peut être exact que les contrats du starter soient verts et

que l'application n'ait pas encore été exécutée. L'honnêteté sur les limites permet de concentrer le travail sur le bon risque.

Les intégrations natives de fournisseurs tiers demandent leurs propres preuves. Un script portable qui fonctionne ne démontre pas qu'un agent propriétaire a été connecté, authentifié et exécuté dans le projet du CLIENT.

10. Que faire après la clôture

Après acceptation, mettez à jour les surfaces canoniques, les références de source et les dépendances qui peuvent réellement être débloquées. Après un rejet pour défaut dans le charter, renvoyez à I3 la correction consolidée. Après invalidation stratégique, reconsidérez le périmètre avec le propriétaire approprié.

Si currentness reste inconnue, maintenez un HOLD ciblé et poursuivez les vérifications sûres. Le CLIENT doit savoir quelles capacités sont utilisables, quels gates restent ouverts, qui les traite et quand un nouveau résultat sera revu.

Une bonne clôture est assez courte pour être comprise et assez précise pour être falsifiable. Si une personne sceptique et absente de l'implémentation ne peut pas retrouver la preuve, l'état n'est pas encore suffisamment reconstructible.

Suite : chapitre 07, l'exemple complet d'une application de gestion de club.

Sources et portée

- Intégrité de freeze et de terminal
- Cycle de vie des capacités
- Validation exécutable
- Gates de release du framework
- Support et distribution v1.4
- Vérificateur de release

L'acceptation appartient au produit, au candidat et aux autorités réels. Ces exemples ne confèrent aucune permission supplémentaire.

07 — Exemple guidé : une application de gestion pour un club

> Cet exemple est **fictif et pédagogique**. Il ne décrit pas une application déjà développée, un backend généré automatiquement ou une preuve qu’un modèle d’IA a livré un CRUD accepté. Dans un projet réel, les exigences, les sources, la DB, les permissions et les preuves doivent provenir du produit adopté.

Dans ce chapitre

1. Le produit demandé par le club
2. Découvrir les questions qui changent le résultat
3. Installer la couche de contrôle, pas l’application
4. Première capacité : gestion des membres
5. Du Planner à I3 : un ordre de travail complet
6. Ce qu’il faut tester avant d’accepter
7. Deuxième capacité : cotisations et paiements
8. Évaluer l’ensemble du parcours
9. Premier cycle avec des données fictives
10. Enregistrer et acheminer un défaut
11. Les dépendances du deuxième cycle
12. Exercice final : récupérer le projet

1. Le produit demandé par le club

Imaginez une petite association locale. Une personne administrative gère les membres, cotisations, activités et inscriptions dans plusieurs feuilles de calcul. La nouvelle application doit permettre de retrouver une fiche, savoir qui a payé, conserver les opérations historiques et ne pas accepter plus d’inscriptions que la capacité d’une activité.

La première version sera utilisée en interne. Le CLIENT décide de ne pas ajouter pour l’instant de paiement en ligne, d’application mobile séparée, de facturation réglementaire, de notifications automatiques ou d’intégrations complexes. Cela limite les risques et permet de vérifier rapidement les comportements essentiels.

Cette description ne fixe pas encore le langage, la DB, le serveur ou les permissions exactes. Le Planner confirme l’usage réel et les règles de produit. L’ingénierie choisit une solution proportionnée en tenant compte du code existant, de la sécurité, du coût et de la maintenance. Installer le framework n’implémente aucune de ces fonctionnalités.

Un ordre de capacités plausible est identité des membres, cotisations, activités et inscriptions. Les cotisations dépendent d’une identité stable ; les inscriptions fondées sur le statut financier dépendent de cotisations correctes. Cet ordre protège les interfaces et évite de programmer des règles de métier contradictoires.

Il ne faut pas confondre ces dépendances avec la nécessité de multiplier les agents. Un Planner peut conserver la stratégie pendant qu'un Controller prend en charge l'ingénierie. Des spécialistes sont utiles lorsqu'il existe un travail réellement séparable ou une revue indépendante pertinente.

2. Découvrir les questions qui changent le résultat

Le Planner demande : qui utilisera le logiciel, combien de membres existent, l'application doit-elle fonctionner sans internet, quelles données personnelles sont nécessaires, comment les sauvegardes seront-elles réalisées et quand la première version doit-elle être démontrée ? Les réponses peuvent modifier le stack, les interfaces et le plan.

Une règle proposée est : «Un administrateur peut désactiver un membre, mais les paiements antérieurs doivent rester disponibles aux personnes autorisées». Cette règle doit être confirmée. La désactivation est-elle réversible ? Le membre doit-il disparaître de toutes les recherches ou seulement du filtre des actifs ? Quelles données doivent rester visibles ?

Autres points : une cotisation en retard empêche-t-elle une inscription ? Les paiements partiels comptent-ils ? Une annulation libère-t-elle immédiatement une place ? Une liste d'attente existe-t-elle ? Peut-on corriger un paiement et qui en a le droit ? Le Planner ne transforme pas des suppositions en règles actuelles pour avancer plus vite.

Domaine	Premier comportement proposé
Membres	Créer, modifier, rechercher, désactiver et conserver l'historique.
Cotisations	Enregistrer les paiements, identifier les retards et consulter les opérations.
Activités	Définir nom, horaire, capacité et places disponibles.
Inscriptions	Ajouter/annuler selon la capacité et les règles de cotisation.
Hors périmètre	Paiements en ligne, app mobile, notifications massives, tableaux de bord complexes.

La table reste une proposition tant que le CLIENT ne l'a pas confirmée. Le besoin de données durables suggère un profil `STATEFUL`, mais cela n'établit pas une politique de confidentialité, un schéma ou une permission de production. Ces éléments sont reconstruits dans l'adoption réelle.

3. Installer la couche de contrôle, pas l'application

Dans un dépôt nouveau et non gouverné, le starter v1.4 peut créer une couche `.agentic-sdlc/`. Depuis la racine du framework, le PowerShell suivant est un exemple :

```
& "python" "tools\scaffold_project.py" --target "..\club-app" --profile "STATEFUL" --p
```

Le résultat initial est `HOLD / UNTRUSTED_CONTEXT`. L'installateur ne crée pas automatiquement une interface ou une DB, ne configure pas d'LLM et n'autorise pas de campagne. Le chapitre 01 explique les commandes de diagnostic et les limites de ce scaffold.

Si le dépôt `club-app/` contient déjà un Product State ou un contrôleur canonique, ne réinstallez pas le starter par-dessus. Utilisez le précontrôle en lecture seule pour reconstruire les références et préparer une éventuelle adoption sans écraser l'autorité existante.

Le Planner et I3 inspectent ensuite les sources, l'inventaire, les données, les exigences et l'environnement. Ils préparent une spécification d'adoption, passent les contrôles de preflight, établissent des références cohérentes et démontrent que le contexte peut être récupéré par une nouvelle session.

Le CLIENT confirme les priorités, contraintes et choix fonctionnels. L'ingénierie propose le stack, les sauvegardes, les accès et les jalons datés. Une campagne matérielle ne devient autorisée qu'après adoption suffisante et charter explicite. Si la source de vérité est inconnue, un HOLD ciblé protège le travail dépendant.

4. Première capacité : gestion des membres

Les paiements et inscriptions ont besoin d'un identifiant de membre fiable. Commencer par les membres est donc une décision de dépendance produit, non une préférence pour une technologie. Un charter pourrait demander :

> Un administrateur autorisé peut créer une fiche, la rechercher, la modifier et la désactiver, sans détruire les paiements et inscriptions historiques qui doivent être conservés.

Le charter définit interfaces, données, propriété, erreurs attendues et gates. Il ne demande pas au CLIENT de concevoir le schéma SQL ou les noms des classes. I3 choisit l'implémentation, gère les tests, les corrections et les revues dans la portée autorisée.

Les contrôles peuvent inclure identifiants en double, données invalides, lecture après écriture, persistance après redémarrage, recherche actifs/inactifs, relations historiques et accès non autorisé. Si l'application possède une interface, le parcours utilisateur complet doit être exercé sur le candidat intégré réel.

Une capture de la fiche créée ne prouve pas que l'enregistrement est durable. Un test de validation des champs ne prouve pas que la personne peut retrouver la fiche. Les gates unitaires, d'intégration, de DB, physiques et perceptuels répondent à des questions différentes.

Si des données réelles existent déjà, l'ingénierie doit préserver l'état actuel et prévoir sauvegarde, migration et rollback adaptés. Les données fictives du manuel ne représentent pas la base de l'association et ne doivent jamais être présentées comme preuves d'une adoption réelle.

5. Du Planner à I3 : un ordre de travail complet

Un message de campagne peut rester concis tout en donnant les références nécessaires. Dans une véritable exécution, il faudrait remplacer cet exemple par les sources et IDs actuels :

Goal: Livrer la capacité de gestion des membres dans le charter autorisé.

Context: Reconstruire les exigences confirmées, le Product State, le dépôt, le commit et la DB actuelle.

Constraints: Conserver l'historique et les permissions ; ne pas développer cotisations, notifications ou fonctions hors périmètre.

Execution gates: Tester réellement création, recherche, édition, désactivation et persistance après redémarrage ; traiter les entrées invalides, les doublons et les accès interdits. Faire une revue indépendante du candidat intégré.

Done when: Fournir l'identité exacte du candidat, preuves physiques, résultats des contrôles, rapport de revue, gates ouverts et terminal stratégique honnête.

I3 peut diviser le travail en tâches, résoudre des bugs, modifier la structure interne, relancer les tests et intégrer un nouveau candidat sans demander un charter à chaque étape. Si le double clic sur «enregistrer» crée deux fiches contrairement à une règle d'unicité, c'est un défaut à réparer dans cette campagne.

Lorsque des actions ont un coût ou un risque matériel, le Controller définit un inventaire, une méthode, des limites et des conditions d'arrêt pour l'unité de travail. Cela ne l'autorise pas à utiliser un fournisseur payant ou à introduire une fonctionnalité externe non demandée.

6. Ce qu'il faut tester avant d'accepter

Prenons un membre synthétique `MEMBRE-001`. Le tableau décrit ce qu'une future application devrait démontrer. Ce n'est **pas** une preuve que ces tests ont déjà été exécutés :

Étape	Action utilisateur	Résultat observable attendu
1	Ouvrir le programme comme administrateur.	Le mécanisme d'accès autorise le rôle prévu.
2	Créer `MEMBRE-001` avec des champs fictifs valides.	Un seul enregistrement apparaît avec un identifiant stable.
3	Rechercher par identifiant autorisé.	La bonne fiche apparaît sans exposer des données privées.
4	Modifier un téléphone fictif.	La nouvelle valeur est visible après enregistrement et relecture.
5	Fermer puis rouvrir l'application.	Les données persistent dans le chemin réel de DB/recovery.
6	Désactiver le membre.	Il n'apparaît plus comme actif, mais son identité historique reste.

Étape	Action utilisateur	Résultat observable attendu
7	Consulter l’historique avec le bon rôle.	Paiements et inscriptions conservés restent disponibles.
8	Essayer une entrée invalide ou un doublon.	L’application refuse clairement sans corruption de l’état.

Pour chaque étape, notez l’entrée, le résultat attendu, l’observation, l’environnement et le commit. Un HTTP 200 ne suffit pas à établir la persistance ; une relecture indépendante est nécessaire. Si le produit promet la durabilité après redémarrage, le test doit réellement redémarrer le processus concerné.

Si deux employés utilisent l’application simultanément, il faut tester les interactions qui peuvent entrer en concurrence. Un test séquentiel ne prouve pas que deux inscriptions simultanées respecteront le dernier siège disponible. Les cas de test dépendent des risques et de l’architecture réelle.

Utilisez des données fictives pour les captures publiques. Ne publiez pas noms, coordonnées ou paiements réels. L’évaluation perceptuelle relève d’une personne compétente sur un candidat représentatif, séparément des contrôles de sécurité et de données.

Si l’application n’a pas encore été construite, les étapes sont `NOT RUN`, pas `PASS`. Le manuel décrit une méthode pour produire des preuves ; il n’a pas exécuté les fonctionnalités de ce club fictif.

7. Deuxième capacité : cotisations et paiements

Une fois la gestion des membres acceptée, le CLIENT demande l’enregistrement des cotisations. Le Planner confirme périodes, échéances, paiements partiels, corrections, rôles et doublons. Un statut «à jour» doit provenir de règles et de données cohérentes, pas d’un calcul improvisé dans chaque écran.

Une politique possible est : «Un paiement saisi par erreur est annulé par une écriture d’audit ; la transaction originale n’est pas effacée». Le CLIENT décide de cette règle. Le Planner l’enregistre ; I3 implémente persistance, validation, idempotence, contrôle des permissions et récupération.

La répétition d’une requête, à cause d’une connexion instable ou d’un double clic, ne doit pas produire un deuxième paiement lorsque le contrat exige une seule opération. Les tests doivent exercer la frontière où le doublon peut réellement survenir, pas seulement une fonction isolée.

Si le produit traite de l’argent réel, les gates de confidentialité, d’autorisation, de journalisation et de publication doivent être renforcés en proportion du risque. Le starter ne contient pas de service de paiement et n’autorise pas l’usage d’un fournisseur externe. Cet exemple sert à penser les données et les preuves.

Les activités et inscriptions viennent ensuite, lorsque l’identité et le statut de cotisation sont fiables. Inutile d’ajouter spontanément microservices, cloud, application mobile ou système complexe de rôles. Chaque mécanisme supplémentaire doit répondre à un besoin ou à un risque observé.

8. Évaluer l’ensemble du parcours

Après cet exercice, vous devriez pouvoir distinguer les décisions du CLIENT, les exigences confirmées par le Planner, l’exécution contrôlée par I3, la revue indépendante et l’acceptation finale. Vous devriez aussi expliquer pourquoi une validation verte du framework ne signifie pas que la gestion du club est prête.

L’enjeu n’est pas de mémoriser chaque nom de JSON, mais de comprendre où se trouve la vérité actuelle, quel acteur peut la changer et quelle preuve confirme un comportement. Une architecture qui génère beaucoup de documents mais ne démontre pas l’enregistrement d’un membre n’a pas livré la première capacité.

Ces principes s’appliquent à d’autres produits : API, réservation, application éducative ou outil interne. Les gates nécessaires varient selon les données, utilisateurs et conséquences, mais la séparation entre intention, autorisation, implémentation, preuve et jugement reste pertinente.

Une comparaison avec la programmation directe dans un chat doit rester honnête. Un petit prototype peut être plus rapide en conversation libre ; un produit long peut gagner en continuité et en qualité de revue. Les bénéfices doivent être mesurés avec des travaux réels, non proclamés comme une supériorité universelle.

9. Premier cycle avec des données fictives

Inventez «Personne de test A» et un numéro de contact manifestement fictif. Confirmez avec le CLIENT les champs requis et l’identifiant unique. Le Planner exprime les critères de produit et l’ingénierie choisit les contraintes de stockage, transactions et gestion des erreurs.

Exécutez le parcours complet : création, liste, recherche, modification, redémarrage, relecture, désactivation et consultation de l’historique autorisé. Notez chaque résultat observé. Une capture de la première création n’est que le début de la démonstration de persistance.

Ajoutez des cas négatifs : identifiant répété, champs invalides, panne contrôlée de DB pendant l’écriture et soumission répétée. Les messages et effets exacts dépendent des règles confirmées, mais l’application ne doit pas produire silencieusement des données contradictoires.

Si plusieurs opérateurs existent, examinez les actions simultanées : deux personnes réservent la dernière place ou modifient le même enregistrement. Un test isolé ne permet pas de conclure que la concurrence est sûre. Le Controller choisit des scénarios représentatifs du risque réel.

Le CLIENT peut ensuite signaler que certains champs sont difficiles à comprendre ou que la désactivation manque d’explication. Le Planner classe le retour : défaut d’un comportement convenu ou nouvelle règle de produit. I3 corrige les problèmes couverts sans ouvrir une campagne stratégique supplémentaire.

10. Enregistrer et acheminer un défaut

Supposons que la désactivation de «Personne de test A» fasse disparaître les paiements historiques. Si leur conservation était exigée, c’est un bug dans la campagne courante. Notez candidat, étapes, données synthétiques, observation et gate concerné.

Le Planner renvoie les falsificateurs à I3. Le Controller recherche la cause dans les relations de persistance, corrige, ajoute une régression, exécute le readback après redémarrage, intègre un nouveau candidat et obtient une revue indépendante pertinente. Il n'ouvre pas une campagne distincte pour chaque symptôme.

Maintenant, le CLIENT demande si un membre inactif peut récupérer son ancien numéro en cas de réactivation. Si cette règle n'avait jamais été décidée, le Planner la confirme avant de l'intégrer à la stratégie. On ne peut pas appeler bogue le choix entre deux comportements qui n'avaient pas été spécifiés.

Le manuel ne doit pas fabriquer de preuve à partir de cette histoire. Tant qu'aucune application réelle n'a été développée, les tests restent `NOT RUN`. Le but est d'apprendre à identifier les autorités, données et observations nécessaires à une acceptation honnête.

11. Les dépendances du deuxième cycle

Les cotisations ont besoin d'un membre stable et de règles explicites sur les paiements partiels, doublons, échéances et corrections. Le Planner utilise la capacité précédente réellement acceptée comme base. I3 implémente sur les interfaces et sources actuelles, pas sur une version mémorisée d'une ancienne conversation.

Avant les inscriptions, vérifiez que le statut des cotisations est produit de manière actuelle et consommable. Si la règle d'éligibilité utilise une dette mal définie, l'API et la UI pourraient donner des résultats contradictoires. Le chemin critique sert à prévenir ces incohérences, pas à justifier davantage d'agents.

La livraison incrémentale permet au CLIENT de tester les membres avant d'investir dans les cotisations. Les retours réels peuvent faire découvrir des exigences nouvelles ; leur effet est plus facile à comprendre quand les capacités ont des interfaces et preuves claires.

N'ajoutez pas spontanément des mécanismes techniques coûteux. Les microservices, services cloud, notifications et paiements externes sont des options, pas des objectifs. La première version doit avant tout résoudre des tâches concrètes avec une sécurité et une maintenance proportionnées.

12. Exercice final : récupérer le projet

Imaginez que le travail s'interrompt aujourd'hui. Demain, un nouveau Planner arrive sans lire le chat précédent. Le dépôt et le runtime autorisé doivent indiquer si la gestion des membres est acceptée, quel candidat et quelles preuves l'établissent, quelles décisions sur les cotisations restent ouvertes, qui possède la campagne et quelle action peut suivre.

Si deux campagnes se déclarent actives sur la même ressource, le nouveau responsable réconcilie l'autorité. Si un rapport est marqué `CURRENT` mais pointe vers une source ancienne, il ne devine pas. Si le témoin de DB est absent, il conserve un HOLD ciblé et poursuit les vérifications sûres.

Classez les événements rencontrés : nouvelle décision métier, bogue dans le charter, panne d'environnement, permission externe manquante ou invalidation stratégique. Chacun suit une route et un propriétaire adaptés. L'architecture réussit lorsqu'il est possible de faire ce classement à partir des sources durables.

Le résultat de l'exercice n'est pas la présence de `.agentic-sdlc/`, ni un nombre de

commits. C’est la capacité d’un produit à accomplir un parcours réel et de l’équipe à en retrouver la preuve après une interruption, sans transformer le CLIENT en coordinateur technique.

Fin de l’exemple : revenez à l’index des chapitres pour approfondir un thème ou consulter les sources techniques.

Sources et portée

- Exemple synthétique minimal
- Fork et bootstrap
- Découverte, exigences et livraison
- Limites du CLIENT
- Standard des charters
- Guide public v1.4
- Précontrôle de gouvernance existante

Ce scénario est pédagogique. Il ne représente ni une application déployée, ni une fonctionnalité acceptée, ni une intégration externe démontrée.